

跟着紅隊走一遍護網演習

每年總有一段時間，一批人守着屏幕盯告警，另一批人對着同樣的目標掃描、嘗試、橫向、截圖。前者是防守的藍隊，後者是攻擊的紅隊，他們共同出演的活動叫護網，也就是 [護網演習](#)。

這篇寫的是紅隊在一次護網演習裏的完整作業流程：演習開始前手裏有甚麼、怎麼把目標摸清、從哪裏進去、進去之後往哪站、往哪走、最後怎麼交差。每一步都寫到能據以判斷的顆粒度：這件事為甚麼要做、做的時候看甚麼、做錯了會怎樣、防守方在對面看得到甚麼。技術手法各自掛在它出現的那個階段上，不單獨成章。

預設的讀者是有運維或開發基礎、但沒有做過攻防的人，也照顧已經在演習裏打過一兩輪的人。所以文中不從零解釋域名、端口、帳號這些詞，但凡出現一個有確切定義、後文會反覆用到的概念，都會登記成詞條並在第一次出現處給出引用。

文中凡引用外部資料，都以腳註寫明出處。來源以攻擊方技術的公共知識庫為主：MITRE ATT&CK 的技術條目、廠商與監管機構的公開通告、以及 RFC 一類的標準文本。一處說法若找不到可靠出處，文中就不把它寫成已查證的事實。

最後有一句必須寫在最前面。本篇描述的動作，全部以獲得授權為前提。沒有授權而對真實系統做同樣的事，性質是刑事犯罪，與演習無關。紅隊與入侵者的分別不在手法，在於有沒有那一紙授權。

1 | 演習開始前，紅隊手裏有甚麼

紅隊在演習開始前拿到三樣東西：一份授權、一份演習規則、一份目標清單。三樣東西裏，目標清單最含糊，它可能只寫着參演單位的名字和幾個必須拿下的靶標，其餘要打哪裏、怎麼打，全靠紅隊自己判斷。

這一段時間通常有三週左右。對防守方來說這是加固期，對紅隊來說這是情報準備期。準備工作分兩頭：一頭是把目標單位的公開信息翻一遍，另一頭是把自己的攻擊基礎設施搭起來。兩頭同時進行，所以下面先說紅隊手裏那三樣東西各自長甚麼樣，再說基礎設施為甚麼必須提前數週就位。

1.1 | 三樣東西各自長甚麼樣

授權 是紅隊全部動作的邊界，所以第一件事是把它的一項一項讀清楚。一份能用於演習的授權至少寫明七項：可攻擊的目標範圍，細到域名、IP 段與系統清單；時間窗的精確起止；允許的行為，例如掃描、口令嘗試、投遞釣魚郵件；禁止的行為，例如拒絕服務與刪改數據；允許造成的影響上限，例如業務中斷不得超過多久；數據的處理約定，即拿到的數據能不能複製出來、演習結束後要不要銷毀；以及聯絡人與應急中止方式。

最後一項最容易被忽略，它卻是紅隊出事時唯一能用的通道。真實演習裏紅隊會碰到目標網段上跑着不在清單裏的系統、或者某一台機器明顯不屬於目標單位這種情形。正確做法是當場停手，把它記進待確認清單，經聯絡人向組織方書面確認之後再動。這一步拖節奏，卻是唯一不會讓演習變成案件的做法。

演習規則 決定分數怎麼算，所以第二件事是把計分方式倒推成行動優先級。規則一般規定六項：計分方式，即成果分與過程分各佔多少、每個 **靶標** 值多少、同一目標被多支隊伍拿下時先提交者得分；提交窗口的具體時段；證據的格式要求；紅線清單；扣分項，通常包括越界訪問與影響生產；以及爭議的裁定流程。

其中「先提交者得分」這一條會直接改變紅隊的策略。它意味着一個入口的價值不只看它通向哪裏，還要看它多久會被別人先拿走。後文講紅隊節奏時說「速度比精巧更重要」，根子就在這一條上。

第三樣是目標清單，它的顆粒度差別很大。最粗的一種只給單位名稱與主域名，紅隊要自己把 **攻擊面** 摸出來；最細的一種給到具體系統與 **靶標** 的位址。兩種情形下情報那一頭的工作量相差數倍，所以開工之前先看清清單細到甚麼程度，再決定備戰期怎麼分配人力。

1.2 | 開打之前要把基礎設施搭起來

另一頭的工作與情報無關，是把紅隊自己要用的東西準備好。這些東西統稱攻擊基礎設施，包括攻擊用的服務器與跳板、中轉代理、用於釣魚的域名與郵箱、以及 **命令與控制** 服務端。

服務器與跳板的選型要看三件事：位址與目標單位的關係，靠近目標所在運營商的位址不容易被地理封鎖規則誤傷；供應商的實名與付款方式，這決定買來的資源會不會在第一週就被追溯；以及被封之後的更換成本，因為目標一旦開始封來源位址，紅隊每天換一批跳板是常態。

域名與證書要在開打前數週就備好，原因是域名的年齡與歷史本身就是一道門檻。剛註冊的域名、沒有收發歷史的發信域，在郵件閘道那裏的待遇明顯更差。證書同理，簽發與部署都要時間。

郵件那一頭要配三樣東西：發信域的 SPF、DKIM、以及 DMARC 記錄。三樣配齊不等於郵件一定進收件箱，但缺一樣就幾乎必然被攔。信譽要靠時間和收發行為養出來，所以這件事辦不齊的原因不是紅隊偷懶，是它本來就要花數週。

命令與控制 服務端要做備份通道。只有一條通道的紅隊，在通道被發現的那一刻就全線失聯，所以常見做法是同時準備好幾條，分別走不同的協議與不同的前置，一條斷了還有下一條。前置的作用是讓流量看起來落在一個無害的位址上，詳見站穩那一章。

代理與隧道要準備多級中轉與出口位址池，因為紅隊的每一次掃描與每一次登入都會在目標那裏留下來源位址，單一出口很快就會被拉黑。釣魚站點與憑據回收的通道也在這一頭，它要能在投遞之後存活足夠長的時間。

把這些併到一起，結論是基礎設施決定開打第一週能走多遠。演習開始之後再申請域名、再買服務器、再等信譽，這條路在演習的節奏裏走不通。

1.3 | 準備工作是兩條並行的線

情報線的產出是資產地圖與入口清單，下一章展開；基礎設施線的產出是能用的跳板、能發信的域名、以及通了的多條通道。兩條線的交集在開打當天：情報線給出「打哪裏」，基礎設施線決定「從哪裏打」。

對防守方來說這三週是加固期。同樣的三週，紅隊在把自己的入口變多，藍隊在把自己的入口變少。兩邊用同一段時間做相反的事，這也是為甚麼演習剛開始那幾天雙方的差距最大。

1.4 | 真實演習是盲打

還有一件事要在心裏接受：真實演習是盲打。題目不會告訴你目標用了甚麼框架、有幾個入口、哪個環節最薄弱。能拿到的只有一個單位名稱和幾個域名。從這一點上說，護網和以明確題面為前提的比賽是兩種活動。

資訊不足對策略的影響是連鎖的。第一，情報階段必須先廣後深，先把能找到的資產全部鋪開，再挑少數幾條深挖；第二，行動由入口清單驅動，沒有清單就沒有行動順序，全隊會退化成各自亂試；第三，要接受大量嘗試沒有結果。外界以為紅隊在演習期間是隨便進出，實際情況是多數嘗試都沒有結果，能把一條路走到底的隊伍是少數。

1.5 | 紅藍對抗下的處境

紅藍對抗 不是單方面的技術練習。對手就守在同一張網裏，每一次掃描、每一次登錄都可能被記錄下來，裁判則會把 **紅隊** 提交的成果與防守方的日誌放到一起核對。這意味着 **紅隊** 要達成的目標不只是打進去，還要打得能被證明。

「能被證明」在技術上有一串直接後果。它要求紅隊在突破的同時保留自己能複現這條路徑的材料，要求每一步操作都留下可核對的時間戳，也要求紅隊在手上有能通過審核的成果之前，不能把操作記錄清掉。這一串後果在交作業那一章收口。

1.6 | 防守方在備戰期本可以做的事

同一段備戰期，藍隊有四件事做了收益很高。一是把自己單位的資產盤一遍並確認歸屬，特別是沒有人認領的那部分，因為紅隊的入口多數就在那裏；二是把確認無用的資產下線或收斂暴露面，外部掃到的東西少一件是一件；三是檢查 **邊界設備** 的補丁與管理口令，但邊界設備數量大、型號雜，這件事在多數單位裏排不進前三；四是監控外部對自己資產的掃描行為，情報階段的掃描量大，這是備戰期裏唯一能提前發現紅隊動作的機會。

2 | 摸底：把目標摸清

信息收集 是整條流程裏最枯燥也最決定成敗的一步。這一步做得好，後面每一步都有方向；這一步做得糙，後面就只能靠運氣反覆試。它要回答的問題只有兩個：目標有多少扇門，以及哪幾扇最容易推開。

2.1 | 先數門

紅隊 動手的第一件事不是攻擊，是弄清目標到底有多少扇門。做法是對目標單位的 **攻擊面** 做一次全面測繪：從一個主域名出發，找出所有子域名、解析出對應的 IP、再掃描這些 IP 上開放了哪些端口、跑着哪些服務。

這一步的關鍵詞是「全部」。漏掉的那些才是有價值的部分。被打穿的單位多半不是被正面突破，而是被一條沒人管的測試線路、一個忘了下線的舊後台、或者開發人員自己搭在某個公網 IP 上的服務帶進去的。這些資產在詞條裏的名字是 **影子資產**，它們的共同點是沒有人為它負責，所以沒有人給它打補丁、看日誌、做加固。

影子資產的具體形態可以列成六種，它們的共同點是「曾經被建起來、現在沒有人管」。一是測試與預發佈環境，它們為了方便通常直接開在公網上，用的是與正式環境同一套程式，但防護鬆得多；二是舊版系統的後台，新系統上線之後舊入口常常只是從首頁撤了連結，服務還在跑；三是開發與運維人員自己搭的服務，它們不在資產台賬裏，因為走申請流程太慢，或者本來只是臨時用一下；四是已經停止使用但仍然解析着的域名，以及指向已經被回收的位址，後者意味着同一個位址可能已經屬於別人；五是外包交付之後沒有完成交接的系統，廠商走了，本單位沒有接手；六是雲上開出來的實例與儲存桶，它們由業務部門自己開，不走本單位的採購與上線流程。後三種在近年的公開事件裏出現得最多，因為它們連「到底有沒有這台機器」都要先查出來。

雲與軟件即服務的暴露面要單獨看，判斷方式與自建機房不同。要查的有四項：對象儲存有沒有被設成公開可讀寫；雲主機的安全組有沒有放開高位端口；介面金鑰有沒有被提交到公開的代碼倉庫；以及軟件即服務的租戶設定有沒有對外開啟共享。這四項的共同點是不依賴目標的技術水平，只依賴有沒有人檢查過默認設定。找子域名有五類來源，每一類的產出形態與盲區都不同，所以要幾類一起用、互相印證。

第一類是 [憑證透明度日誌](#)。查一個已知域名，能拿到所有在憑證裏寫過它的域名。它的產出最乾淨，因為憑證裏的域名欄位是簽發時由申請人自己填的，寫進去的都是真實使用過的名字。盲區是不用公開憑證的系統完全不在其中。

第二類是 [被動 DNS](#)。它記的是解析歷史，所以能撈出已經下線但仍在記錄裏的域名，也能看出兩個域名是否指向過同一個位址。它與第一類的盲區正好互補：憑證管用過的名字，解析歷史管用過的位址。

第三類是搜索引擎與代碼倉庫。前者收了大量被隨手發佈出去的內部位址，後者常常在設定檔、說明文件、提交歷史裏留下內部域名與測試帳號。這一類的產出雜，但經常能檢到別的來源都沒有的東西。

第四類是字典與置換爆破。拿一個字典去解析，命中的就是活着的。這一類的缺點是噪音大且會留下查詢記錄，所以節奏要壓住。

第五類是域名服務器的設定洩漏，包括區域傳送沒有關、以及設定檔被備份到公網。這一類一旦命中就是一次拿全，但命中率低。

名字找齊之後要把位址那片也還原出來。做法是先確定域名的位址屬於哪個 **自治系統號**，再把那個號對應的網段拉出來。這樣做有兩個好處：一是能找到沒有域名、只有位址的資產；二是能判斷某台機器到底是不是目標自己的。後者直接關係到 **授權** 邊界，所以寧可少打也不要打錯。

端口與服務的掃描要決定兩件事。一是掃全端口還是只掃常見端口。全端口慢，但漏掉的端口往往是故意放在高位的那種；常見端口快，適合在第一輪鋪開的時候用。二是要不要掃 UDP。多數人只掃 TCP，而一些管理與監控協議跑在 UDP 上，所以它是常見的遺漏區。

內容分發網絡 要單獨處理。從外部解析到的位址可能是節點的位址，把它當成目標資產去掃，既浪費時間也可能掃到與目標無關的機器。判斷方法是看解析結果是否落在已知的節點網段，以及不同地域的查詢是否返回不同位址。

這一節給一個推導示例會比較好懂。假設目標的主域名是 example.com。從憑證透明度日誌裏查到一張憑證，它的域名欄位除了 example.com 還寫着 oa.example.com 與 vpn.example.com，一次查詢就多出兩個入口。再拿 oa.example.com 去查解析歷史，發現它三個月前指向過一個已經不在解析表裏

的位址，而那個位址落在目標自己的自治系統號下，於是一個已經下線的辦公系統被重新找了出來。整個過程沒有碰目標的任何一台機器，所以沒有製造任何噪音。

這一節寫的只是骨架。五類來源各自怎麼查、產出是甚麼形態、盲區在哪、以及從名字走到網段、再走到端口與指紋、最後憑甚麼判斷一台機器是目標單位的，這些都另立了一篇專題，寫在同目錄的 `recon.typ` 裏。

2.2 | 從人身上收集

資產之外，**紅隊**還要收集人。要收集的內容包括員工姓名、崗位、內部郵箱地址、以及郵箱的命名規則。來源是公開的：單位的官方網站與公眾號、招聘網站上的崗位描述、技術社區裏的分享與提問、以及各類商業信息查詢平台上的股權與子公司關係。

組織結構那一頭要拿到的是三張圖。第一張是部門與職責，它決定誰手上有甚麼系統的權限；第二張是技術棧，招聘描述裏寫的「熟悉某某中介軟體」這類字樣，往往比從外部掃描猜指紋更準，這一條同時餵給後面認指紋那一節；第三張是人員與對外聯絡位址，它決定後面投遞清單怎麼列。

技術棧那張圖可以從三處拼出來，它們的可靠程度依次下降。第一處是招聘描述，它會直接寫出單位在用的中介軟體、資料庫與雲平臺，而且因為要吸引人，寫得通常比實際情況更具體。第二處是技術團隊對外的分享，包括會議議程、開源項目的貢獻記錄、以及技術博客，它們會暴露具體的版本與架構取捨。第三處是公開的招標與採購公告，它會寫明買了哪一套產品與服務，這一類的價值在於它能覆蓋到那些不對外提供服務的內部系統。三處合起來的結果比從外面掃描猜指紋可靠得多，因為這些話是目標自己說出來的。郵箱命名規則是可以推出來的。先從公開的對外聯絡位址看格式，常見的是姓名全拼、姓加名首字母、工號、以及英文名加數字這幾種。再拿一條已知的姓名與位址對上去，就能確定是哪一種，然後按同一規則生成其餘人的位址。推出來的要標成推測，因為一條寫錯的位址在投遞那一步就是一次暴露。

還有一類來源是歷史洩漏的憑據。這類資料的價值不在於它一定能用，而在於它是口令那條路的起點：知道一個人在別的網站上用過甚麼樣的口令，就能推出他本單位的口令大概長甚麼樣。使用這類資料之前要確認它與本次授權的關係，拿不準的不要用。

這些信息有三個用途。一個是投遞清單，知道誰在甚麼崗位，才知道用甚麼理由能讓他把附件點開。一個是口令字典，內部系統的口令往往由姓名、工號、年份拼成，知道命名規則就等於拿到了字典的一部分。還有一個是找出第三方的入口，單位的軟件供應商、外包運維公司、代管數據的合作方，這些單位的防護通常比目標本身弱。

2.3 | 認出對面是甚麼

摸到資產之後，要給每一項資產認出**指紋**。一個 Web 系統跑在甚麼中介軟體上、用了甚麼框架、版本是多少，一台設備是哪個廠商的哪款型號，這些判斷會直接決定後面用甚麼手段。

指紋的載體有九類。響應頭裏的伺服器欄位與頭部的順序，頭部順序是中介軟體自己拼出來的，不容易改，所以它比版本號可靠；Cookie 的名字與結構；默認頁面與報錯頁，中介軟體在出錯時往往暴露自己的名字與路徑；靜態資源的路徑與命名規律；網頁圖示的雜湊值，同一個產品的圖示雜湊是固定值，所以它常用於判斷這是哪一套管理平台；憑證的簽發者與主題欄位；網絡棧的行為特徵；前端打包產物的結構；以及端口的默認值。

除了一台一台認，還有一個方向是認防護。目標有沒有[網頁應用防火牆](#)，用的是哪一款，決定了試探的節奏。判斷方法包括故意發一個大概率會被攔的請求看回應、看回應頭裏有沒有防護產品留下的欄位、以及比較同一請求在不同位址上的回應差異。

認資產歸屬這件事要寫重一點，因為它同時是技術問題與法律問題。一台機器是不是目標單位的，可以有四條證據：它的位址落在目標的[自治系統號](#)或備案網段裏；它的憑證主題欄位寫着目標單位的名字；它的頁面內容與目標單位的業務有關；以及它的域名註冊資訊指向目標單位。四條至少要中兩條，只中一條的要標成待確認。真實演習裏打錯資產是嚴重的違規，而且這類錯誤往往發生在掃描結果最多的那幾個小時。

判斷防護的存在要允許反向的結論，因為有三種情形會讓人誤判。一是回應裏出現的產品欄位可能是對方故意加的，用來讓攻擊者以為有防護而放棄；二是同一個回應在不同位址上不一致，這說明中間有一層按位址分流的設備，而不是目標系統本身的差別；三是攔截頁面可能來自目標的上層運營商或者雲服務商，這種攔截不由目標控制，它的規則也不會隨目標的業務變化。把這三種排除掉之後再下結論，比看到一個欄位就寫進清單可靠。指紋

判斷必須允許不確定。同一個回應頭可能來自中介軟體，也可能是防護設備偽造的，所以要把每一條指紋標成已驗證或推測，推測的結論不能用來決定高風險動作。

指紋的價值可以用一句話說明：漏洞是按組件分的，不知道對面是甚麼組件，就不知道該試哪個漏洞。一個已經公開了兩年但目標一直沒打補丁的漏洞，其價值往往不低於一個剛出現的未知漏洞，因為前者有現成的利用工具，後者要自己從頭做。

2.4 | 這一步的產出

摸底做完，紅隊手上應該有兩份東西。一份是資產地圖，把目標單位的域名、IP、端口、系統、負責人線索串成一張表。另一份是按可行性排序的入口清單，每一條寫明入口是甚麼、需要甚麼條件、成功了能拿到甚麼。

資產地圖的欄位要能回答五個問題：這是甚麼，在哪，屬於誰，跑着甚麼，誰在維護。最後一欄往往空着，而空着的那幾行就是後面優先看的地方。

入口清單的欄位比資產地圖多，每條要寫：入口的位址、對應的 **弱點** 類型、利用需要的前提，即要不要帳號、要不要有人配合、成功之後能拿到甚麼、這一步會花掉多少可被發現的痕跡、以及這條路以前有沒有人用過。最後一欄決定它值不值得試，因為已經被別人打過的入口，防守方往往已經補上了。

排序用的是一個粗模型：預期收益乘以成功率，再除以暴露成本。預期收益看拿下之後能通向哪；成功率看需要的前提是否已經具備；暴露成本看這一步會不會在防守方那裏留下明顯的痕跡。三個量都只能估，所以模型的作用不是算得準，而是逼著全隊把三個量分開想，不要只盯著收益。

情報的可信度建議分三級標在清單上，因為它直接決定這條情報能不能用於高風險動作。一級是已經在授權範圍內驗證過的，例如紅隊自己訪問過的系統，這一級可以直接用；二級是有多個獨立來源互相印證的，例如憑證記錄與解析歷史同時指向同一個域名，這一級可以用於試探；三級是單一來源或者靠推測得到的，例如按命名規則推出來的郵箱位址與猜出來的目錄名，這一級只能在低風險動作裏用。把三級混在一起的最直接後果是，紅隊會在某一次失敗之後不知道該懷疑情報還是該懷疑手法。第二份東西比第一

份重要。演習的時間有限，所有入口都試一遍是不現實的，只能按成功率與收益排序，從最上面一條往下打。另外，情報的可信度要標在清單上。已驗證的與推測的要分開，因為推測的入口一旦失敗，浪費的不只是時間，還有這條路上本可以拿到手的東西。

2.5 | 防守方在情報階段本可以做的事

這一階段的攻防幾乎是單向的，紅隊在暗處，藍隊能做的是把暗處照亮。有四件事收益最高。一是用攻擊者的視角對自己掃一遍，把外部能看到的資產列出來與自己手上的清單對比，差集就是 **影子資產**；二是核對憑證簽發記錄，發現不該有的簽發就說明有資產脫管；三是把無人認領的系統下線或加一層存取控制，這一件事拖得越久越難做；四是監控外部對自己資產的掃描，大規模的憑證查詢與字典解析都是有特徵的，而且這些動作發生在演習開始之前，是整個演習裏最容易提前發現紅隊的機會。

3 | 打點：從攻擊面進去

打點 指把入口清單上的某一條變成一個能執行命令的位置。這一章按入口的來源分成三條路：對外服務的系統與設備、人、以及第三方。三條路在

紅隊的實際排期裏是並行的，因為它們的成敗互相獨立，一條走不通不影響另一條。

3.1 | 從互聯網邊界進去

這一條路針對的是對外提供服務的系統。它在紅隊的入口清單上排在最前面，原因不是它最容易，而是它不需要任何人配合，可以二十四小時不停地試。

網頁應用的手法可以歸成五類，它們的共同點是同一件事：把使用者能控制的資料，送進一個會被當成程式碼或指令處理的位置。

第一類是文件上傳。一個上傳介面能變成命令執行，要三個環節同時成立。一是檔案要能被寫進伺服器上可訪問的目錄；二是那個目錄裏的檔案要能被當成程式解釋，判斷依據是目錄的解析設定，而不是檔案的副檔名；三是執行那個檔案的身份要有足夠權限。三個環節缺一個，上傳就只是上傳。紅隊判斷自己能不能利用的時候，就是在分別驗證這三個環節。

第二類是注入。按被注入的位置分，常見的有資料庫查詢、作業系統命令、模板、以及表達式四種。四者的構造不同，但可利用性來自同一個前

提：程式的撰寫者把「資料」與「指令」拼在了同一個字串裏。所以紅隊要判斷的是兩件事：這個位置的使用者輸入能不能影響到指令的結構，以及影響之後能不能拿到回顯。沒有回顯的注入仍然可以利用，但成本高得多。

四種注入要分開說，因為紅隊要先知道對面是哪一種，才能決定值不值得投入。資料庫注入的判斷重點是有沒有回顯：有回顯的直接讀資料，沒有回顯的要靠條件真假或者時間延遲去問，而這兩種問法都要求同一個請求可以重複發送。作業系統命令注入的判斷重點是輸入有沒有流進一個會執行命令的函數，它的特徵是分隔符號可以直接改變命令的結構。模板注入的判斷重點是輸入有沒有進入模板的求值過程，它與前兩者的差別在於可利用的深度大得多，因為模板語言本身能訪問宿主語言能訪問到的東西。表達式注入常見於需要對輸入做計算的框架，判斷方式與模板注入接近。第三類是 **反序列化**。它的觸發點不在輸入本身，而在資料流經的邊界。判斷一個系統有沒有這類問題，要先找有沒有「一端寫入、另一端還原」的位置，再確認還原時接受的類型是不是可以由寫入端指定。這兩件事都成立的時候，危險性取決於該程式載入了哪些可用的類。

第四類是 [越權訪問](#)。它與前三類的性質不同：前三類是在找程式的技術缺陷，這一類是在找權限檢查的缺失。它的兩個常見形態是改參數與直接訪問。改參數是把請求裏的物件標識換成別人的，看伺服器有沒有核對歸屬；直接訪問是繞過前端，直接請求一個本該只有管理員能到的位址。判斷方法是把每一個介面都問一遍：這個介面在伺服器那一側，有沒有核對過「你有沒有資格做這件事」。

第五類是 [服務端請求偽造](#)。它出現在「由使用者指定位址、由伺服器去取」的功能上。判斷的關鍵是伺服器能訪問到甚麼，而不是那個功能本身看起來多麼無關緊要。

除了這五類，還有一類不屬於技術缺陷，是認證與業務邏輯上的問題，例如驗證碼可以被繞過、密碼重設流程可以被推測、下單與退款的金額由前端決定。這一類沒有通用判斷方法，只能把業務流程當成攻擊面重新走一遍，每一步都問「這一步的前提是誰保證的」。

¹見 [OWASP Top 10](#)。

OWASP 維護的網頁應用風險清單把這一類問題做了歸類，¹ 紅隊在入口清單上做初步分類時可以直接用它的類目，但真正決定能不能利用的仍是具體系統的實作。

網頁應用之外的另一半是 **邊界設備**。拿下的價值比拿下一台業務伺服器高，原因有兩重。一是邊界設備的管理面通常同時連着外網與內網，拿下它就等於同時有了外網的落腳點與進內網的跳板；二是它的策略配置本身就是情報，一份防火牆規則能說明目標的網絡分區與信任關係。

邊界設備這一類裏，常見的六種入口形態不同。防火牆與負載均衡設備管的是流量放行，它們的管理面往往單獨開一個端口或一個位址；虛擬專用網絡設備管的是遠端接入，它的價值在於一個帳號就能進到內網；郵件閘道管的是郵件，它同時是釣魚那條路的正面對手；堡壘機管的是運維入口，它把憑據集中在自己手上；上網行為管理與偵測設備管的是出網與流量還原，它們的記錄本身就是防守方復盤時的材料；以及工業控制網絡的各類隔離設備，它們的問題通常不在產品本身，而在投產之後沒有人再動過配置。六種的共同點是它們都位於信任的邊界上，所以任何一台被拿下，紅隊得到的都不只是一台機器。邊界設備長期補丁落後是普遍現象，原因有四個，而且它

們互相加強。一是這類設備數量多、型號雜，一個單位有幾個廠商的十幾種設備是常態，逐台更新的工作量遠大於伺服器；二是很多型號已經停產，廠商不再發布補丁，只能靠外層的存取控制頂着；三是設備的更新窗口很難找，重啓一次可能中斷整棟樓的網絡，所以沒有人願意批這個窗口；四是設備的管理口令常由外包或整合商掌握，本單位既不知道口令也不知道有沒有其他人在管。這四個原因合起來的結果是，紅隊在邊界設備這條路上的成功率多年來一直不低。

堡壘機單獨說一句。它既是運維的入口也是憑據的倉庫，所以在入口清單上，一台堡壘機的優先級通常高於它後面管着的十台伺服器。

還有一類入口不含任何技術漏洞，是配置錯誤。常見的有五種：對象儲存被設成公開可讀寫；資料庫或快取直接監聽公網；管理後台沒有任何存取控制，只要知道位址就能進；介面沒有做認證就把內部資料吐出來；以及裝好之後從未修改的默認頁面。這一類的處理時間通常只有幾分鐘，它的存在不取決於目標的技術水平，取決於有沒有人在做資產盤點。

這一節的五類網頁手法、邊界設備的六類、以及上線那一段的完整展開，另有一篇專題篇：寫在同目錄的**對外服務與邊界設備的漏洞利用**。它按

同樣的分工寫，每一類都落在「值不值得投入」上：五類手法各寫機制、判斷依據與失敗模式；六類設備各寫管理面開在哪、怎麼認出來與配置洩漏能拿到甚麼；上線那一段補上本篇沒有展開的存活期、以及被清掉之後的處置。

3.2 | 從人身上進去

社工釣魚 這條路的價值在於它繞開了全部的 **邊界設備**。防火牆按位址與端口放行，郵件閘道按內容特徵過濾，但這兩者管的都是流量。釣魚郵件本身是正常郵件，打開它的動作發生在員工自己的電腦上，而那個位置在設計上就位於防線之內。所以邊界設備越密，這條路的相對性價比越高。

一條完整的釣魚鏈路有四步，每一步都有各自的成敗條件，紅隊要能分別判斷自己卡在哪一步。

第一步是投遞，也就是讓郵件進到目標的信箱。要處理的問題有三個。一個是發信域的信譽，這一項在第一章已經說過，需要提前養。一個是繞過郵件閘道的着力點，閘道的判斷維度包括寄件人信譽、附件類型、連結指向的域名年齡、以及郵件的結構特徵，紅隊能調整的就是後面幾項。還有一個是投遞的節奏與數量。一次性投給幾百人，只要其中一個人上報，全單位都

會收到預警，後面幾週都別想再投。所以實際做法是分批、每批少量、先投給最不容易上報又最有權限的人。

第二步是落地，也就是郵件到得了對方的信箱之後，內容以甚麼形態呈現。常見的形態有壓縮包、捷徑檔案、帶巨集的文檔、腳本檔案、光碟映像，以及一條指向仿冒登入頁的連結。這些形態的差別在於觸發條件。有的要在打開之後再點一次，有的要在系統彈出警告時選擇允許，有的則完全不需要互動。觸發條件越多，被攔截或被放棄的機率越高，所以紅隊在這一步上要判斷的是目標終端的防護強度，而不是挑一個看起來最厲害的形態。

連結那一種要單獨說，因為它的門檻最低，但每一處細節都能讓人起疑。要做對的有四件事：域名要與目標的域名足夠像，但同時不能是目標域名的子域，否則會落進對方自己的憑證與郵件防護範圍；憑證要有效，瀏覽器位址欄不能出現警告；頁面要與真實登入頁一致，包括報錯的文案；以及提交之後要真的把使用者送進真實系統。最後一件最容易被忽略，因為紅隊拿到憑據之後往往直接去用，忘了把使用者送回去，而一次失敗的登入會直接變成一次上報。第三步是點開，這一半靠技術一半靠話術。決定成敗的通常不是附件做得好不好，而是那封郵件說的事情在對方的工作裏是否合理。

所以前面收集的組織結構與崗位資訊在這一步兌現：知道對方最近在處理甚麼事，才知道用甚麼理由能讓他動手。MITRE ATT&CK 把這一步的各類做法歸在釣魚技術下，²

第四步是上線，也就是從被打開的那台終端連回紅隊自己的控制端。前三步成功而這一步失敗的情形很常見，原因是終端上的防護軟件在行為層面比郵件攔道嚴得多，而且它看到的是程序在做甚麼，不是郵件長甚麼樣。這一節的細節在下一章講。

釣魚之外還有幾條路。一條是語音，冒充技術支持或上級，讓對方在電話裏把驗證碼念出來。一條是即時通訊，冒充同事或供應商發一個檔案。還有一條是直接冒充運維人員，以「來做設備巡檢」這類理由進入內部網絡。這幾條路對紅隊的組織能力要求高，但它們繞開的環節比釣魚更多。

釣魚的性價比在一件事上很突出：它的成功與目標的技術水平相關性低。一個技術很強的單位，也可能有一個剛入職的行政人員。紅隊在技術那條路上投入一週沒有結果之後，通常會回頭把這條路的優先級調上去。

²見 [T1566](#)。

3.3 | 從供應鏈與第三方進去

這條路的思路是不打目標，打目標信任的人。信任關係在網絡上留了具體的痕跡：防火牆上有一條只對某個位址開放的策略，某個帳號在目標系統裏有一份超出常規的權限，某個流程裏有一個不需要再審批的環節，或者兩個單位之間拉了一條專線。找到這些痕跡，就找到了紅隊可以借用的入口。

常見的第三方有六類。軟件供應商，它的更新通道本身就是一條能進目標內網的路徑；外包運維公司，它手上有目標系統的帳號；數據代管方，它與目標之間有數據交換；雲與整合商，它持有目標的控制台權限；分支機構或子公司，它的防護通常比總部弱，而它與總部之間是通的；以及各類合作方，比如支付、物流、客服這些與業務直接打通的介面。

這條路上最有名的公開案例是二零二零年披露的軟件供應鏈事件。攻擊者先拿下軟件廠商的更新系統，再把後門隨正常的更新包分發給廠商的客戶，於是客戶在安裝一次例行更新時自己把後門裝了進來。美國網絡安全與基礎設施安全局為此事發布的通告編號是 AA20-352A，³ 這份通告值得紅藍雙方都讀一遍，因為它把攻擊者的完整手法鏈寫了下來，包括他們怎麼用口令

³見 [該通告](#)。

猜測拿到初始存取、怎麼繞過多因素認證、以及怎麼在受害單位的所在地租用伺服器，讓自己的流量看起來不那麼突兀。

這條路對紅隊的意義與前面兩條不同。走技術那條路，紅隊拿到的是目標系統上的一個執行位置；走這條路，紅隊拿到的可能是對方早就有的、而且被信任的存取權。於是在防守方那一側，這個動作看起來不是攻擊，是一次正常的登入。

它的代價也在這裏。第三方往往不在本次授權的範圍之內，所以紅隊在動手之前必須先確認：目標單位的授權能不能覆蓋到這個第三方，以及需要不需要組織方另行協調。按第一章講的原則，範圍之外的一律先停手、先書面確認。

3.4 | 漏洞的取捨

1day 與 Nday 的分界不在漏洞本身，而在目標修沒修。這個劃分方式決定了紅隊的排期，因為兩者的可用工具與被發現的風險完全不同。

一個漏洞從被公開到被大規模利用，中間有一段時間差。剛公開的幾天裏，知道這個漏洞的人少，防護設備的規則庫也還沒跟上，所以紅隊的可乘

之機最大；但這幾天裏可用的利用工具也少，紅隊要自己把利用過程做出來，成本高。到了公開一兩週之後，工具開始流傳，防護設備也有了規則，紅隊的成本降下來，被發現的機率升上去。再往後就是 Nday 的區間：補丁早已發布，但總有相當比例的系統沒有修，而防護設備對這類攻擊的特徵識別已經很成熟。

所以三類漏洞的用法不一樣。Nday 用來擴大戰果，因為它的成本低、可以批量試，代價是動靜大；1day 用來打那些明顯加固過的目標，因為防護設備還不認識它；未知漏洞留給最關鍵的一擊，因為它只能用一次，用掉之後對方就會查明白並補上。

紅隊判斷一個漏洞值不值得用的時候，問的是三個問題：這個系統上有沒有更省事的路；用這個漏洞會留下多大的痕跡；以及這個漏洞能在多少台機器上重複使用。三個問題的答案合起來，比漏洞本身的評分更能決定它用不用。

一個漏洞從公開到真的能用，中間要過的關比多數人以為的多。第一關是前置條件：要不要先有一個帳號、要不要能訪問到某個內部位址、要不要

目標開啟了某個可選功能。第二關是要不要有人配合，例如誘導一次點擊或者等一次重啓。第三關是能不能重複利用，只能觸發一次的漏洞在演習裏用處有限，因為它不能被用來驗證環境，一旦失敗就沒有第二次。第四關是這條利用鏈有多長，每多一步就多一次被發現的機會，也多在日誌上留一處痕跡。紅隊在排期時把這四關過一遍，比看漏洞本身的評分更能決定它值不值得用。最後一句是統計上的提醒。公開的入侵統計裏，直接利用對外服務漏洞進去的比例近年一直在上升，這也是紅隊把這條路放在清單最前面的原因，⁴但它同時意味着防守方對這條路的監控投入最多，所以紅隊在這條路上的每一步都更容易被記錄。

把對外服務漏洞之外的幾條路放在一起比較，可以得出一個對排期有用的結論：不需要漏洞的路，包括口令、配置錯誤、以及釣魚，它們的單次成功率通常低於漏洞，但它們不依賴目標的組件版本，所以不會因為對方升級一次就全部失效。紅隊在演習裏的常見做法是兩類並行：一邊按入口清單試漏洞，一邊同時跑口令與釣魚，看哪一頭先出結果。

⁴見 [T1190](#)。

3.5 | 弱口令這條老路

口令噴灑、撞庫、以及憑據填充這三個詞經常被混着用，但它們的三件事完全不同，而且防守方的偵測規則也要分開寫。

口令噴灑是用少量常見口令去試大量帳號，每個帳號只試一兩次。它的設計意圖就是避開鎖定策略，因為鎖定通常按同一帳號的連續失敗次數計數。

撞庫是拿別處洩漏的帳號口令組合來試目標系統，它利用的是人在多個系統上重複使用口令這個習慣。它的流量特徵與口令噴灑正好相反：來源集中、帳號分散、每個帳號的口令都不一樣。

憑據填充是把已經拿到的憑據拿去做二次使用。它的位置在整個流程的中段，是內網擴散的主要動力。

除了這三種，還有一種最省事的入口是默認憑據。設備出廠時的帳號口令、中介軟體的默認管理帳號、以及安裝之後從來沒有人改過的初始口令，都屬於這一類。它與前面三種的區別是不需要猜，只需要知道對方用甚麼產品。

口令猜測的命中率取決於字典，而字典是可以推的。第二章收集到的姓名、工號、以及單位的名稱與年份，按常見的拼接規則可以生成出一份針對性很強的字典。這一份字典的效果通常遠好於通用字典，原因很簡單：通用字典裏的口令，單位的口令策略多半已經把它屏蔽了，而由本單位人員姓名與年份拼出來的口令，看上去又複雜又符合規則。

單位的口令策略是可以反推的，推出來之後字典的命中率會明顯上升。要觀察三點。一是最短長度與複雜度要求，這通常能從註冊頁面的提示文案或者報錯訊息裏看出來；二是更換週期，從員工離職與新人入職的公開時間、以及常見的口令命名習慣可以推出大致的輪換規律；三是歷史口令的關聯性，不少單位的規則是在舊口令後面加一個數字，於是新舊口令的差異極小，這一條在拿到一份舊憑據之後特別有用。**多因素認證**是這條路上真正的障礙。它在口令之外再要一個因素，所以知道口令也登不進去。但它並不是不可繞，而且繞法通常不在於破解第二個因素。AA20-352A 裏記錄的一種做法是：攻擊者拿到了用於簽發身份憑證的私鑰，於是直接偽造出一張已經通過認證的憑證，整個驗證過程被從中間跳過去了。另一類做法出現在網頁端，攻擊者做一個與目標登入頁完全一樣的頁面，在使用者輸入口令與動態碼的那一刻把兩者都收走，再用動態碼在有效期內登入真實系統。這兩類做

法的共同點是：它們都沒有破解第二個因素，而是讓驗證通過的那個瞬間發生在攻擊者手裏。

對紅隊的實際啟示是，當目標的對外入口已經全覆蓋多因素認證，繼續在口令上投入的收益就很低了，應該把資源移回釣魚與對外服務漏洞那兩條路。判斷覆蓋率的方法是從登入流程本身看：一個入口在輸入口令之後是否必然要求第二個因素，以及在多個入口之間是否存在不一致。

為甚麼防守方明知這條路存在還要反覆強調它：因為它是唯一一條不需要任何新漏洞、不需要任何技術突破、也不依賴目標業務的路。只要有一個帳號用了弱口令，前面所有的技術投入都可以繞過去。

3.6 | 防守方在打點階段本可以做的事

打點階段的防守有一個特點：紅隊嘗試的次數遠多於成功的次數，所以防守方有一堆可以抓的機會，問題在於有沒有在看。有六件事收益最高。一是把對外服務的組件版本做成清單並盯著補丁，這一件事的效果直接對應 Nday 那條路；二是給所有對外入口補上存取控制與認證，特別是那些以為「沒人知道位址」的後台；三是把多因素認證覆蓋到所有能從外部訪問的入口，並檢查有沒有一條路徑能繞過去；四是針對口令噴灑與撞庫分別寫偵測

規則，而不是合成一條「登入失敗過多」；五是給郵件閘道加上對新註冊域名的檢查，這一條直接對應釣魚的第一步；六是對第三方通道做一次梳理，把「誰可以進我們的網」這張表列出來，因為紅隊打第三方之前，防守方往往連有這條通道都不清楚。

4 | 站穩：立足點、提權與憑據

打點成功之後，紅隊拿到的通常只是一條命令的執行結果，而不是一個可以長期使用的落腳點。這一章講的是把一次性突破變成穩定陣地的三件事：把通道做穩、把權限做高、把憑據攆起來。

4.1 | 先要一個能反覆回來的點

「能執行一條命令」與 **立足點** 是兩件事。前者是漏洞利用那一刻的產物，它隨着那次會話的結束而消失；後者要求紅隊在沒有那條漏洞的情況下，仍然能主動回到這台機器上。這個差別決定了紅隊能不能在這台機器上做後續的事，因為後續的每一步都需要多次往返。

已經有執行位置卻建立不起立足點，通常是被三種情形斷掉。一是那一條命令只能執行一次，例如一次性的反序列化觸發；二是對方重啓了那個服

務，漏洞利用的效果不復存在；三是 **端點偵測與回應** 在命令執行之後把那個程序清掉了。三種情形的對策不同：第一種要在那一次機會裏把後續通道一次做完；第二種要選一個不會被例行重啓帶走的入口；第三種要把動作做得不被行為層認出來。

命令與控制 通道有幾種常見形態，它們的差別在暴露面與存活期。反向連接是被控端主動連回控制端，它穿過出網策略的能力取決於目標允許哪些出網流量；直連是控制端連被控端，它在目標不允許入站時不可用；域前置與內容分發網絡前置讓中間設備看到的主機名與實際處理請求的主機名不一致，所以按名字做的封鎖會失效；域名系統隧道把資料藏在查詢與回應裏，它的優點是幾乎所有網絡都放行域名查詢，缺點是頻寬極低，只能用來傳小指令；走雲服務與協作平臺的通道把流量混進正常業務流量，它最難被按流量特徵識別，代價是紅隊要依賴一個自己控制不了的第三方。

單獨說一句 **域前置**。它依賴大型內容分發網絡同時承載大量客戶域名這個特性，而這個特性近年在多數服務商那裏已經被明確禁止，所以它是一種會過時的手法。紅隊判斷它能不能用的依據是服務商當前的策略，不是它過去的名聲。

通道的挑選依據是目標的出網策略，所以要問三個問題。目標允許哪些協議與端口出網，這一條決定能不能直連；出網流量有沒有被解密與還原，這一條決定偽裝要做得像不像，而不是做得隱不隱蔽；以及目標有沒有對出網建立過基線，這一條決定心跳的間隔能壓到多短。三個問題的答案決定了紅隊在速度與隱蔽之間的位置，而這個位置在演習期間會隨防守方的處置而改變，所以要留有換通道的餘地。不管用哪種通道，都有一個共同的偽裝問題要處理，也就是 **隱蔽通道** 那一節講的事：選甚麼流量做載體，取決於目標網絡裏甚麼流量必須放行。

心跳與抖動 決定通道能不能存活得久。固定間隔的回連在流量統計上是一個明顯的節律，防守方建立一段基線之後很容易把它挑出來，所以被控端回連的時間間隔要加隨機量。代價是指令下發的延遲變大，而演習的時間有限，所以這個參數要在隱蔽與速度之間取一個值。

通道之外，被控端本身也要藏。**進程注入** 是為此服務的一種做法。它借的宿主是已經在運行的進程，所以它的痕跡落在跨進程的記憶體操作上，而這一類事件正是端點防護最敏感的幾類之一。

[白利用](#) 借的宿主不同，它用的是系統自帶的、本來就有正當用途的工具。這樣做的好處是紅隊不需要處理檔案本身的特徵，因為被呼叫的工具是系統的一部分，按檔案信譽攔不住。可用的工具清單有公開整理，⁵

這兩條路都要面對 [端點偵測與回應](#)，而它看的四個維度恰好覆蓋了這兩條路的痕跡：一個程序是誰啟動的、它有沒有對別的進程做記憶體操作、它有沒有建立一個不該有的子程序、以及它的命令列參數是甚麼。四個維度合起來，使這兩條路比自帶一個程序更容易被發現。

因此樣本繞過終端防護的思路，實際上是三個方向，而且它們的難度依次上升。第一個方向是讓靜態特徵不匹配，也就是改動檔案的形態讓基於特徵的檢查認不出來；這個方向的成本最低，但只對靜態檢查有效。第二個方向是讓行為不落在已知的規則裏，例如不用自己建立子程序、改用已有的程序、把長時間的行為拆成多段；這個方向要對目標的防護規則有了解，而且一旦規則更新就會失效。第三個方向是不落地，讓執行內容只存在於記憶體或一個已經被信任的宿主進程裏；這個方向最難被靜態檢查發現，但它本身在行為層留下的痕跡最重，因為跨進程的記憶體操作是端點防護最敏感的幾類事件之一。

⁵見 [LOLBAS](#)。

把這三個方向放在一起看，可以得到一個對紅隊有用的結論：繞過能力不是一個開關，而是一組取捨。每一個方向都在「不被靜態檢查發現」與「不被行為分析發現」之間做交換，而防守方的防護越強，能交換的空間越小。

立足點做好之後還有一件事是檢查它穩不穩。做法是隔一段時間主動確認一次通道是否仍然有效，並準備好第二條通道。只有一條通道的紅隊，在通道被處置的那一刻就等於從這台機器上消失了，而重新建立一次立足點的成本遠高於維持兩條通道。

4.2 | 提權的意義

提權 指把當前拿到的執行身份換成權限更高的身份。為甚麼要做這件事，取決於當前身份能碰到甚麼。一個以普通使用者身份運行的程序，通常讀不到本機的憑據存放區、改不了系統設定、也裝不了會被開機自動執行的東西。而這三件事正是把一次性落腳變成穩定陣地的前提。

它同時是憑據那條路的入口。一個低權限帳號的憑據只能用在這台機器上，而管理員的憑據往往能用在一批機器上。所以提權的收益不是「這台機器上能做的事變多」，而是「能用在這台機器上的憑據的價值變高」。

Windows 上常見的提權面有五類。一是服務設定缺陷，包括服務執行檔的路徑沒有加引號、服務執行檔所在目錄對普通使用者可寫、以及服務本身的權限過大。二是目錄權限缺陷，關鍵在於有沒有可寫的目錄落在高權限程序會讀取的路徑上。三是計劃任務，它與服務那類問題同源，差別只是觸發方式。四是核心驅動，這一類屬漏洞，需要對應版本。五是與令牌有關的一類，也就是從一個已有的高權限程序中取得令牌，見令牌竊取。

Linux 上常見的提權面也是五類，名字不同但邏輯相似。一是帶有特殊權限位元的可執行檔，任何使用者執行它都會以檔案擁有者的身份運行；二是命令提權工具的設定，配置裏允許某些命令以高權限執行，而這些命令往往能順帶執行其他命令；三是定時任務，包括腳本本身可寫與腳本所在目錄可寫兩種；四是核心漏洞；五是能力位元，把原本屬於管理員的部分能力單獨授予了一個程序。

兩邊的共同邏輯是一句話：提權的問題幾乎都出在「高權限的東西讀取或執行了低權限使用者能改的東西」。所以紅隊的做法不是背工具，而是把本機上所有「以高權限運行、且路徑或參數落在可寫位置」的東西列一遍。

這十類提權面每一類的原理、怎麼枚舉、怎麼判斷、失敗模式與痕跡，寫在同目錄的專題篇 **提權** 裏。本篇這一節只保留判斷鏈上的位置與這一句指針，機制與操作留在那一篇。

提權失敗有幾個常見原因，它們與技術水平關係不大，更多是判斷上的疏漏。一是提權面確實存在，但需要一次重啓或者一次登出才生效，而重啓在演習規則裏可能屬於影響生產；二是提權成功但會留下明顯痕跡，例如新增一個服務或者改動一個系統目錄的權限，這一類動作在端點防護那裏是高度可疑的事件；三是當前身份已經能讀到想要的東西，紅隊卻仍然花時間去提權，這一類屬於目標沒想清楚；四是提權之後沒有立刻取走憑據，等到防守方處置之後才想起來，那時機器已經不在手上了。提權成功之後第一件事不是繼續橫向，是 **憑據轉儲**。原因是這台機器上現在能讀到的東西，價值遠高於這台機器本身的算力，而這些東西一旦錯過，可能要重新提權一次才能再拿到。

4.3 | 憑據才是真正的戰利品

憑據 指能證明身份的東西，包括口令、口令的雜湊值、身份票據、金鑰、以及各類應用自己保存的認證資訊。紅隊在內網裏的主要產出不是「拿

下了多少台機器」，而是「手上有多少份可用的憑據」，因為憑據可以重複使用、可以在多台機器上使用，而且不需要再找漏洞。

憑據在本機上的存放位置可以分成八處，紅隊按這個順序去找效率最高。一是程序的記憶體，這是最容易被遺忘也最容易被取走的一處；二是本機的帳號資料庫，它存的是口令的雜湊值，不一定能還原成口令，但可以用於某些認證方式；三是身份票據的存放區，票據與口令的差別是它可以直接被使用；四是組態檔與部署腳本，運維人員把口令寫在腳本裏是常態；五是瀏覽器，它保存的登入狀態本身就是憑據；六是運維與管理工具，它們為了免密操作，本地往往存有大量目標機器的憑據；七是程式碼倉庫，提交歷史裏的憑據即使後來刪掉了也還在；八是自動建置與部署系統，它持有的金鑰權限通常很大，因為它要能部署到所有環境。

從中取出的東西有三種形態，用途不同。明文口令可以直接用於登入，範圍最廣；雜湊值要用於那些以雜湊值本身作為憑證的認證方式，或者用於離線推導口令；票據則可以直接拿來訪問域內資源，不需要知道口令，但有有效期。

一份憑據拿到手之後，紅隊做的是一個循環：用這份憑據去試其他機器，在能進去的機器上再找新的憑據。這個循環的速度遠快於逐台找漏洞，因為它不需要等待漏洞利用的條件成熟，只需要一次登入。術語表把這一步叫做 **憑據填充**。

憑據拿到手之後第一件事是判斷它的範圍，因為範圍決定它值不值得保留。本機帳號的憑據只能用在這台機器上，它對橫向沒有幫助，但可以用來讀取這台機器上更多的東西；本機管理員的憑據能用在同一批設定相同的機器上，這一批有多大取決於部署方式，逐台手工裝出來的環境裏它很小，批量映像出來的環境裏它很大；服務帳號的憑據價值最高，因為服務帳號通常不會被輪換，而且它的權限是為了解服務跑起來而配的，往往比它實際需要的更大；域帳號的憑據範圍最大，但它的每一次使用都會留下可以被關聯起來的記錄。憑據這條路在日誌上的表現與漏洞那條路完全不同。用漏洞進去的痕跡是「一個不該有的程序被執行了」；用憑據進去的痕跡是「一個帳號在一個不該出現的時間從一個不該出現的位置登入了」。後者在多數單位裏的告警優先級更低，因為它看起來像一次正常的運維操作，而且內網裏每天的跨機器登入量本來就很大。這也是紅隊更願意走這條路的原因之一。

憑據有兩個會失效的維度，紅隊在排期時要算進去。一個是時間，票據與工作階段都有有效期，所以拿到它之後的動作要在有效期內完成，否則要重新取得。另一個是口令輪換，如果目標在演習期間改了管理員口令，那麼已經拿到的明文口令會失效，而票據可能仍然有效，因為票據的簽發不依賴口令的當前值。

還有一個風險是帳號被鎖。用憑據反覆嘗試會觸發鎖定策略，而鎖定一個管理員帳號會同時影響目標的正常運維，這在演習規則裏通常算影響生產。所以紅隊在用憑據試機器之前，要先確認目標的鎖定策略，並且控制每個帳號的嘗試次數。

這一節只寫到判斷層。口令與憑據的完整展開另有一篇專題篇：[口令與憑據](#)。那一篇寫的是三種猜口令的手法各自留下甚麼流量、字典怎麼按目標的公開資訊造出來、怎麼判斷目標有沒有鎖定策略並算出每個帳號的嘗試預算、默認憑據從哪查、多因素認證的覆蓋率怎麼判斷以及它有哪兩類繞法、本機八處憑據存放位置各怎麼取、憑據的範圍與時效怎麼判斷、以及這一整條路在日誌上留下甚麼。

4.4 | 防守方在站穩階段本可以做的事

這一階段已經不是「有沒有被進來」的問題，而是「能不能在被進來的早期發現」的問題，因為紅隊在這一段做的事都很安靜。有五件事收益最高。一是把端點防護的覆蓋補齊並保持更新，包括那些長期離線或老舊版本的機器，因為它們往往是紅隊選中的落腳點；二是限制管理員身份在普通辦公終端上的登入，讓紅隊拿到一台辦公電腦也換不到管理員身份；三是收斂本機的憑據快取，尤其是運維工具與腳本裏的明文口令，這一件事直接把紅隊的循環速度降下來；四是把內網的跨機器登入做成基線並監控偏離，而不是只盯著失敗的登入；五是保留足夠長的日誌，因為紅隊的動作往往在事後復盤時才看得出來，而日誌保留期短於演習週期的單位，等於自己放棄了復盤的材料。

5 | 進內網：橫向移動與域

內網是紅隊整個流程裏唯一一段不能靠速度取勝的階段。在外面掃描時，被發現的代價只是換一批位址；進到裏面之後，每一次動作都可能讓防守方把已經拿下的落腳點收走。所以這一章的第一個主題是節制，第二個主題才是手法。

5.1 | 內網不是互聯網的縮小版

內網的第一個差別是它分了區。一個單位規模大一點就會有辦公網、生產網、管理網，以及對外服務所在的**隔離區**。四個區之間的策略寬鬆程度不同：辦公網與生產網之間通常管得嚴，辦公網與互聯網之間管得鬆，而管理網與其他各區之間往往有一條允許運維人員通過的通道。紅隊最容易穿過去的就是這一條通道，因為它是為了方便而存在的。

第二個差別是流量方向。互聯網上的流量以南北向為主，也就是客戶端到伺服器；內網裏最大的一部分是東西向，也就是伺服器之間、伺服器與資料庫之間、以及運維工具與被管機器之間的流量。這一塊流量在設計上就很大，所以紅隊藏在裏面比在外面更容易。

第三個差別是出網。內網的機器大多不通外網，要出網得走代理，而代理通常只放行必要的位址與協議。這一條直接決定了**命令與控制**通道要用甚麼形態，也決定了紅隊的跳板要放在哪個區。紅隊用來解決這個問題的技術統稱**隧道與代理**。

隔離區 單獨說一句。拿下隔離區裏的一台機器並不等於進了內網，兩者之間往往還有一次跨區跳躍。反而是拿下 **邊界設備** 的收益更高，因為它常常同時連着兩個區。

5.2 | 內網要再摸底一遍

在外面做過的摸底，進到裏面之後要做第二遍。這一遍的目標與第一遍相同，都是把「有多少扇門」弄清楚，但做法完全不同，差別在 **噪音預算**。

在外面的時候，掃描量大、被記錄也無所謂，因為對方除了封位址沒有別的辦法。在裏面的時候，一次全網段掃描就會產生一連串的異常連接，防守方只要有人在看，幾個小時之內就能定位到源頭機器，而紅隊在那一區的落腳點也就丟了。所以內網的摸底要按「每一步花掉多少噪音」來排。

要收集的內容有六項，按優先級排：本區的網段與存活主機；每台主機上開放的端口與服務；檔案共享的目錄與權限，這一項常常直接給出下一批靶子；身分驗證相關的服務，因為它們決定了能用甚麼方式登入；本機與網域的身份資訊，包括當前帳號的權限與所在的工作群組；以及能從本機看到的其他區的位址，這一項決定跨區的路怎麼走。

低噪音的做法有幾條共用的思路。一是優先讀取本機已有的資訊，而不是主動探測，例如查本機的設定檔、連線記錄、以及已知的主機清單，這些動作幾乎不產生新的網絡流量。二是對必要的探測做分批與限速，把一次全網段掃描拆成多天、每次只掃一小段。三是先問「這台機器上已經有的東西裏有沒有我想要的」，再問「我要不要去掃別的機器」。四是優先用協定本身的查詢功能，而不是用泛用掃描，因為前者在流量上更像正常行為。

本機已有的資訊具體要查甚麼，兩類系統各有一批可以直接讀的東西。Windows 上要讀的是本機所在的網域與信任關係、本機可以訪問的網路位置、共用目錄及其權限、已連接過的遠端資源記錄、以及本機的服務與排程清單。Linux 上要讀的是本機的使用者與群組、可用的私鑰與已知主機清單、掛載的網路儲存、以及各類設定檔裏出現過的位址。這些東西的共同點是它們本來就在機器上，讀它們不產生任何新的網絡流量，所以是噪音預算最低的一批動作，應該在動手之初就做掉。這一節是整篇裏最容易被省略的一節，原因也很現實：它不如漏洞利用有意思，而且它的成果在演習報告裏不好看。但紅隊在內網卡住的時候，多數情況不是因為找不到漏洞，而是因為不知道旁邊還有甚麼機器。

5.3 | 橫向移動靠的是管理工具

橫向移動指從已經拿下的一台機器移動到另一台。紅隊在這一步用的東西有個共同特點：它們都是目標自己裝的、而且是必需的。遠端桌面、檔案共享、遠端程序呼叫、計劃任務與系統服務、以及自動化運維腳本，這五類在一個正常的內網裏每天都在被使用。

遠端桌面是最直接的一類。它的使用痕跡是登入事件與一條持續的連線，所以它的風險不在「能不能用」，而在「用了之後會不會被注意到」。紅隊在意的是目標有沒有對管理員的遠端登入做限制，以及有沒有把管理員的登入限制在管理網段內。

檔案共享本身不是執行通道，但它同時承擔兩個功能：一是把要執行的內容送到目標機器上，二是讀取目標機器上本來就有的檔案。後者的價值常常更高，因為共享目錄的權限配置鬆弛是內網裏的普遍現象，而且讀取共享目錄不會觸發任何執行類告警。

遠端程序呼叫是系統之間呼叫介面的機制。紅隊用它有兩個原因：一是它在流量上與正常的業務呼叫混在一起，二是它的呼叫過程由系統元件完

成，不需要紅隊自己帶程式。缺點是它對呼叫方與被呼叫方的版本與配置有要求。

計劃任務與系統服務是兩種持久化的執行方式。它們的差別在觸發方式與存活期：計劃任務可以按時間或事件觸發，服務則在系統啟動時自動運行。紅隊用它做 **權限維持** 比用它做橫向更多，原因是一次建立可以長期生效。

自動化運維腳本與 **組態管理工具** 是這一段裏價值最高的一類。這類工具的設計目的就是在在一台機器上對很多台機器下命令，所以它天然具備橫向執行的能力，不需要紅隊做任何額外的事。而且它的設定檔與下發腳本裏常常直接寫着高權限帳號，所以拿下它得到的往往不只是通道，還有憑據。

除這五類之外，還有兩個目標值得單獨列出來。一個是虛擬化管理平臺，它管理的是一整片虛擬機，拿到它就等於拿到了那片虛擬機上的所有系統。另一個是備份系統，它保存了所有系統的歷史資料，而且為了還原方便，它的存取權限通常配置得比生產系統更寬鬆。

橫向移動時優先選目標自己已有的工具，理由有三個，而且都是從防守方的規則反推出來的。一是它已經在防護軟件的白名單裏，不需要為它的來源做解釋；二是它不需要落地新的檔案，所以不會觸發檔案信譽與檔案落地的檢查；三是它的正常用途本來就包含遠端操作，所以在流量上與業務混淆，難以按行為判定。反過來說，紅隊自己帶的工具在這三條上都要額外做處理，成本更高，失敗率也更高。這些通道有一個共同的技術特徵：它們都不需要新漏洞。紅隊用的是目標自己配好的、被信任的、每天在被使用的東西。這也解釋了為甚麼內網階段的重點從「找漏洞」變成了「找憑據與找配置」。

從防守方那一側看，這一段的偵測難點在一件事上：一次異常的橫向登入與一次正常的運維登入，在單條日誌上幾乎沒有差別。同樣的帳號、同樣的協定、同樣的成功結果。能被區分出來的東西只有三樣：時間、來源機器、以及這次登入與該帳號的日常基線是否偏離。而這三樣都要求防守方事先建過基線，沒建過基線的單位在演習期間基本看不出來。

5.4 | 域與域控

域與域控是 Windows 網絡裏集中管理身份的方式，也是紅隊在內網階段的終點。要理解為甚麼，先把它結構說清楚。域是一組由同一套身份體系

管理的機器與帳號；多個域可以組成樹，共用一套命名空間；多棵樹可以組成林，互相之間建立信任關係。這個結構的意義是身份可以跨域使用，而信任關係一旦建立，一個域裏的管理員在另一個域裏也可能有權限。

認證的過程分三步，這三步決定了為甚麼憑據能橫向。第一步，使用者向認證服務證明自己是誰，證明的方式是提供憑據，這個過程的結果是一張票據；第二步，使用者拿着票據去要一張針對具體服務的票據，這個過程由認證服務完成，使用者不需要再提供憑據；第三步，使用者拿這張服務票據去訪問目標服務。

這個設計的關鍵在第一步的產物上。**票據**一旦簽發，使用它就不需要再提供口令，驗證它的是服務本身與認證服務之間的信任關係。所以紅隊拿到票據之後，可以直接用它去訪問，既不需要知道口令，也不需要再做一次認證。這就是「憑據橫向」在技術上的來源，也是為甚麼密碼那一條路的終點是票據而不是口令。

域控是這個結構的中心。它保管域內所有帳號與機器帳號的口令雜湊，它簽發票據，它也保存域的組態與信任關係。所以拿下域控的意義不是「多了一台機器」，而是「這個域裏所有身份都已經在手裏」。

拿到域管理員的憑據之後，下一步不是繼續在域內找機器，而是先判斷這個域的邊界在哪。要問的有三件事：這個域與哪些域建立了信任關係，信任的方向是單向還是雙向；本域的管理員身份在對面那個域裏有沒有權限，這一條取決於信任的類型；以及林裏有沒有一個權限更高的組，例如整個林的管理員組。三件事合起來決定紅隊還能不能再往上走一層，因為演習的計分通常按最終拿到的權限層級算，而一個域的管理員與整個林的管理員之間往往還差一步。從一封釣魚郵件到域控的路徑，在公開復盤裏通常長這樣：一次成功的釣魚拿到一個普通員工的會話；用這個會話拿到該員工機器上的憑據；用這份憑據在內網裏試其他機器，找到一台運維人員也用過的機器；在那台機器上拿到運維或管理憑據；用管理憑據找到域控並取得域內權限；最後在域控上取得整個域的身份資訊。這條路徑裏沒有任何一步需要未知漏洞，每一步用的都是目標自己的管理工具。

這也解釋了防守方在域控上投入的原因。防住這條路徑的關鍵不在域控本身，而在中間那幾步：普通員工的機器上有沒有管理憑據、運維人員的帳號有沒有在辦公終端上登入過、以及域管理員的身份有沒有被限制在專用機器上。

5.5 | 站樁與打掃

走到這一步之後，紅隊要處理兩件事：讓已經拿到的權限活到演習結束，以及決定留下多少痕跡。第一件事叫 **權限維持**。

權限維持的落腳點有四類，挑選的標準是「能活多久」與「被發現的機率」。一是新增帳號，它最容易被發現，因為帳號清單是防守方會定期核對的東西。二是計劃任務或系統服務，它比帳號隱蔽，但會被基線比對挑出來。三是中介軟體或應用自身的設定，例如在其中加一段啟動腳本，它最不容易被發現，因為它藏在業務組態裏，而業務組態的變更審計通常比系統組態鬆。四是憑據本身，也就是不裝任何東西，只保留一份能再次進來的憑據，它的存活期等於憑據的有效期，而痕跡最小。

四類的存活期與痕跡是反比的，紅隊的選擇取決於還需要多久，以及這台機器對防守方有多重要。演習後半段通常會傾向第四類，因為那時的目標不是繼續擴大戰果，而是保住已有的分數。

第二件事是 **痕跡清理**。清理的對象包括新增的帳號、留下的檔案、以及操作日誌。這裡有一個取捨必須講清楚：清理到甚麼程度要與舉證的需要一起考慮。紅隊的分數來自提交的證據，而證據要能自證，也就是要能證明

這次操作確實發生在演習時間窗內、發生在這個系統上。痕跡清得太徹底，紅隊自己手上能用於舉證的材料也一併消失了。

實際做法是先把證據固定下來，再清理。固定的內容包括命令執行的截圖、靶標系統的識別資訊、以及時間的證明；清理的範圍則避開這些內容。這個順序不能顛倒，因為清理是不可逆的。

5.6 | 防守方在內網階段本可以做的事

內網階段的防守與前面幾個階段有一個根本差別：紅隊已經在裏面了，所以「不讓他進來」這個目標已經失效，能爭取的只有「早點發現」與「別讓他拿到域」。有六件事收益最高。一是把分區隔離做成真的隔離，而不是只在防火牆上寫了規則、實際上有大量例外；二是給 **特權帳號** 做專用管理機器，讓管理員身份永遠不出現在辦公終端上，這一條直接切斷紅隊從一台辦公電腦走到域控的路；三是把域控與其他機器的日誌集中起來分析，因為單看某一台機器的日誌看不出橫向移動的全貌；四是給身分驗證服務加監控，尤其是對那些不該出現的票據請求；五是佈置 **蜜罐** 與誘餌帳號，它們不會誤報，因為它們不對外提供任何業務；六是定期核對帳號與計劃任務的清單，這一條針對的是權限維持，而且它需要的投入很小。

這一章只寫到判斷層。內網與域的細節另有一篇專題篇展開：[域與內網橫向](#)。那一篇寫的是四個分區怎麼分工、從辦公網到管理網那條允許運維通過的通道怎麼找、五類橫向通道各自留下甚麼痕跡、Kerberos 認證的三步在網絡上分別長甚麼樣、以及從一台辦公終端走到域控的完整路徑。

6 | 交作業：舉證與報告

[舉證與報告](#) 是整條流程裏最不像技術工作、卻最影響成績的一步。演習的計分不看 [紅隊](#) 自己說拿下了甚麼，只看提交的證據能不能通過審核。

6.1 | 證據要能自證

一份合格的證據要同時回答三個問題：證明操作確實發生在目標系統上、證明當時確實擁有相應權限、以及證明這一切都發生在演習時間窗內。所以常見的證據形態是命令執行結果的截圖。

截圖裏要同時出現四樣東西，缺一樣就可能被駁回。一是目標系統的識別資訊，主機名或內網位址，而且要與組織方下發的 [靶標](#) 清單對得上；二是當時使用的帳號；三是當時的時間；四是那條命令的執行結果本身。四樣東西要在同一張圖裏，把兩張圖拼起來的做法通常不被接受。

靶標的標識是最容易出問題的一項。「看起來像靶標」不算，因為裁判要核對的是組織方指定的那一台。所以紅隊在動手之前就應該把靶標清單上的識別資訊記下來，而不是事後從截圖裏找證據。

時間的證明有兩個層次。一個層次是系統顯示的時間，另一個層次是這個時間與提交窗口的對應關係。前者可能被目標系統的時間設定影響，所以紅隊在自己的操作記錄裏也要留一份時間戳，兩者互相印證。

證據的組織方式會直接影響審核效率。常見的做法是按靶標分組，每一組裏按攻擊鏈的順序排，每一張截圖配一句說明它證明了哪一步。這樣做的好處是裁判不需要自己去猜哪張圖對應哪一步，而有沒有這一步說明，往往就是通過與被要求補交的差別。

常見被駁回的證據有五種形態。一是截圖裏沒有靶標的識別資訊，只有一個命令執行的結果；二是截圖裏沒有時間；三是用本地環境的截圖冒充目標系統；四是證據的內容與提交的路徑描述對不上；五是提交的時間超過了規定的窗口。這五種都與技術能力無關，但它們造成的分數損失與沒打下來是一樣的。

提交這件事本身也有節奏。演習的提交窗口通常每天只開若干小時，而證據審核需要時間，所以紅隊的做法是把已經能自證的成果分批提交，而不是攢到最後一天一次交。分批的好處有兩個：一是如果某一批被駁回，還有時間補材料；二是審核方早一點開始核對，也能早一點發現紅隊自己沒注意到的格式問題。最後一天才提交的隊伍，通常會在窗口關閉之前發現證據缺一項而無法補救。最後補一句與技術無關但常被忽略的事：證據要在動手的時候就開始固定，不要等演習結束再回去找。原因很簡單，紅隊在中間那一段幾乎一定會做 **痕跡清理** 的動作，而那些被清掉的日誌與檔案，往往正是事後唯一能證明某一步的材料。

6.2 | 報告與復盤

演習結束之後，**紅隊** 要交一份報告，內容是完整攻擊路徑、每一步用到的 **弱點**、以及每一步在防守方那裏有沒有留下可發現的痕跡。這份報告的讀者不是 **紅隊** 自己，而是參演單位的防守團隊。

一份能用的報告，目錄大致是這樣六節。第一節是結論，用一段話說明從哪個入口進去、最終拿到了甚麼；第二節是攻擊路徑，按時間順序把每一步寫清楚，每一步標明用到的技術與它在公共知識庫裏的對應編號；第三節是入口清單，列出所有被嘗試過的入口以及失敗的原因；第四節是防守方的

可見性，也就是每一步在目標的日誌與告警裏留下了甚麼，哪幾步本可以被發現；第五節是修復建議，按優先級排，而不是按技術難度排；第六節是附錄，放證據與時間戳。

第二節與第四節是這份報告真正的價值所在。第二節回答「我怎麼進來的」，第四節回答「你為甚麼沒有看見我」。多數單位的安全投入偏向第二節，因為修復一個入口是看得見的工作；而第四節帶來的改進往往更大，因為它調整的是發現能力，而發現能力對所有入口都有效。

復盤會通常要回答三個問題，它們對應三種不同性質的缺口。第一個是可見性缺口，也就是某一步明明有日誌卻沒有人看見，這一類靠補規則解決；第二個是流程缺口，也就是日誌存在、規則也告警了，但沒有人負責處置，或者處置得太慢，這一類靠改流程解決；第三個是優先級缺口，也就是問題早就知道、整改也排進了計劃，但一直排在別的任務後面，這一類只能靠管理層調資源。三種缺口的修復成本差很多，所以紅隊在報告裏把它們分開寫，比混在一起列一張長長的問題清單有用得多。報告的價值在於把「怎麼被打穿的」講清楚。對防守方來說，最有用的信息不是某個具體漏洞的利用方法，而是攻擊鏈上那些本該被發現卻沒有被發現的環節：為甚麼一台沒

人認領的測試系統會連在生產網裏，為甚麼一個已經公開兩年的漏洞沒有打補丁，為甚麼一次異常的跨網段登錄沒有觸發告警。紅隊的報告與藍隊的復盤合起來，才是演習留給這個單位的東西。

6.3 | 防守方在交作業階段本可以做的事

這一階段表面上已經結束了，但防守方能做的事決定了這次演習能留下多少東西。有三件事收益最高。一是把日誌的保留期與完整性先確認一遍，因為復盤的深度上限就是日誌能回溯的深度；二是按紅隊報告裏的第四節逐條核對自己當天的告警記錄，如果一條都對不上，那說明不是漏看了某一條告警，而是整個發現鏈路有缺口；三是把「本可以發現卻沒有發現」的環節轉成具體的檢測規則或流程改動，而不是轉成一份停留在文件裏的整改計劃。

7 | 打得不順的時候

這一章講的是紅隊在演習中段的常態。前面幾章描述的是一條走通的路徑，而真實演習裏走不通的時候更多，區分一支隊伍水平的地方就在於它怎麼處理走不通。

常見的阻礙有三類，它們的性質不同，對策也不同。第一類是防禦設備起了作用：掃描被流量設備識別、利用請求被 **網頁應用防火牆** 攔截、上傳的內容被終端防護清除。這一類的特徵是「同一個動作反覆失敗」，對策是換入口而不是加大力度，因為防護規則一旦命中，同樣的動作只會讓來源更快被拉黑。第二類是網絡條件限制：目標機器不出網、資料庫與應用分離在不同網段、橫向端口被主機防火牆關閉。這一類的特徵是「已經拿下的落腳點停在原地」，對策是重新評估這台機器的位置，判斷它在這張網裏能碰到甚麼，而不是繼續在它身上找出路。第三類是 **藍隊** 的處置：發現異常之後封禁來源位址、重置口令、清理入口。這一類的特徵是「已經有效的通道突然失效」，對策是先確認自己的哪一步被發現了，因為處置通常是針對某一條路徑的，不搞清楚原因就重建，新通道會用同樣的方式再被發現一次。

蜜罐 的識別值得單獨說，因為它與前兩類的失敗模式不同：它不是讓紅隊失敗，而是讓紅隊成功得太容易。有四個特徵值得警惕。一是過於乾淨，一台內網機器上沒有任何日常使用的痕跡，沒有文件、沒有登入記錄、沒有業務流量；二是權限過於整齊，一進去就是管理員，而真實環境裏從普通使用者走到管理員通常要幾步；三是資料過於誘人，憑據、設定檔、密碼本擺在顯眼位置，真實環境裏這些東西不會這樣放；四是互動過於配合，系統對

任何輸入都有回應，包括那些在正常系統上應該報錯的輸入。四個特徵中兩個以上同時出現時，合理的選擇是放棄這台機器。

藍隊處置對節奏的影響比多數紅隊預想的更大。一次處置不只是廢掉一條通道，它會讓紅隊失去這台機器上已經收集到的憑據與環境資訊，而重新收集的成本高於第一次，因為此時防守方已經知道有人在裏面。所以在演習裏，「保住已經拿到的」這件事的優先級會隨時間上升。

節奏被打亂之後的重建有一個順序。先停掉所有還在跑的動作，避免繼續暴露更多來源；再檢查其餘通道是否還在，判斷這次處置影響的範圍是一台機器還是一個網段；然後從還沒被觸及的那一區重新開始，而不是回到原來的位址硬頂；最後把這次被發現的原因寫進自己的記錄，因為同樣的原因在演習裏通常會再犯一次。

7.1 | 防守方在這一階段本來能做的事

打得不順是紅隊的視角，換到對面就是防守正在起作用。把前面幾節翻過來看，有四件事最能讓紅隊卡住。一是把處置做成流程而不是靠個人，事先寫清楚發現異常之後誰封、誰改口令、誰保留現場，因為紅隊最怕的不是被發現，是被發現之後在幾十分鐘之內被切斷；二是處置之前先留證據，不

少單位在慌亂中直接重裝機器或者清掉日誌，結果紅隊是被趕走了，這次攻擊是怎麼發生的卻永遠查不清，同一條路下一次還會走通；三是把誘餌布在主幹路上而不是角落裏，蜜罐的價值取決於紅隊有沒有機會碰到它，放在沒人去的網段等於沒有；四是對已經被拿下的機器做一段時間的觀察，而不只是清理，因為清理只解決這一台，觀察才能發現紅隊用同一份憑據還進了哪幾台。

還有一件容易被忽略的事是節奏。紅隊的節奏被打亂之後必然要重新試探，而重新試探會產生一批新的異常流量。所以處置之後的那幾個小時是偵測的黃金窗口，它比處置本身更有價值，而這個窗口在慌亂中最容易被浪費掉。

8 | 紅隊的分工與節奏

一支紅隊小隊通常三到五人。最常見的分工是按工序切：一個人負責情報，把資產、員工、指紋整理成可以直接使用的清單；一到兩個人負責突破，按清單去試入口；一個人負責內網與收尾，接手已經進去的機器，做提權、憑據收集與橫向移動。

這種切法的好處是每一段都有明確的交付物。情報那一段的交付物是入口清單，突破那一段的交付物是能執行命令的位置，內網那一段的交付物是憑據與權限。有了交付物，一支隊伍才不會在「大家都在忙但不知道進度」的狀態裏耗掉一週。

演習的節奏可以細到天。假設是三週的演習，第一週通常是情報與試探：前三天做資產與人的收集，把入口清單排出來；第四天到第七天按清單逐條試，這一段的產出通常是一到兩個能進去的入口，以及一批「確認此路不通」的結論。第二週是集中推進：前半週把進去的入口做穩、提權、收集憑據；後半週用憑據在內網鋪開，目標是找到一條通往域的路。第三週前半段是衝刺，把價值最高的靶標拿下；後半段是守成果與交證據，把所有權限整理成能通過審核的材料，並按提交窗口分批提交。

這個分段在不同單位會有偏移，但有一個規律是穩定的：分數主要產生在第二週，而第三週的收益取決於第一週有沒有把入口清單做出來。這也是為甚麼情報那一段枯燥卻不能省。

大型演習裏同一批目標往往分給多支隊伍，公共目標池裏的目標更是大家爭搶。這種安排下，速度比精巧更重要：一支隊伍花了三天做出的優雅攻

擊鏈，可能不如另一支隊伍兩小時前用弱口令拿下的同一台機器，因為分數只認誰先提交。

這一條在第一章講演習規則的時候已經埋下，它的來源是「先提交者得分」那條計分規則。所以要理解紅隊為甚麼經常選粗糙但快的路，不能只看技術，要看規則怎麼算分。

這也是真實演習與學術研究在節奏上的根本差別。研究追求的是方法的新穎與可複現，演習追求的是在窗口關閉之前把分拿到手。把這兩件事混在一起，會得出一些在演習裏完全行不通的結論，例如「這個手法不夠優雅所以不用」。

8.1 | 防守方在分工與節奏上本可以做的事

紅隊的分工是按工序切的，防守方的分工卻常常是按設備切的，這是兩個團隊在演習裏節奏不同的主要原因。按設備切的問題是每個人只看到自己那一段：看流量的看不見終端上的動作，看終端的看不見身分驗證上的異常。所以防守方在演習之前最該做的一件事是把分工改成按攻擊鏈切，也就是讓同一個人同時盯着「一個入口被打、一台機器被登入、一個帳號被提權」這三件事有沒有接上。

節奏上有一件事與紅隊正好相反。紅隊的分數在演習中段產生，所以紅隊前鬆後緊；防守方的處置窗口卻集中在被發現之後的那幾個小時，所以防守方需要的是隨時能響應，而不是隨時都在忙。把處置能力做成值守而不是靠加班，是這一章能給防守方的建議。

9 | 紅線：甚麼不能做

演習規則裏的紅線不是擺設。拒絕服務攻擊會讓真實業務中斷，刪改數據會造成不可逆的損失，影響生產系統運行可能直接變成事故。這些行為一旦發生，參演單位的損失是真實的，演習的意義也隨之消失。

紅線的具體內容各次演習會有出入，但歸納起來通常是四類：不得造成業務中斷或效能明顯下降；不得修改、刪除、或對外帶走真實業務數據；不得對目標系統做不可逆的變更，包括改設定與裝驅動；以及不得攻擊授權範圍之外的位址。四類的共同點是它們造成的後果都不可逆，而演習的目的是暴露問題，不是製造問題。

有一點容易誤解：紅隊在演習裏拿下的系統，其影響範圍同樣要控制在授權允許的上限之內。例如一次成功的提權如果導致某個服務重啓，而這個

服務在目標單位的關鍵路徑上，這次提權在規則上就屬於越界，哪怕技術上完全正確。所以紅隊在動手之前要判斷的不只是「能不能做」，還有「做了之後對方會不會中斷業務」。

還有一條線在演習之外，第一章已經寫過，這裡只補它與規則紅線的差別。規則紅線管的是「允許範圍內不可以做的事」，授權邊界管的是「根本不在允許範圍內的東西」。前者的判準是影響程度，後者的判準是有沒有寫在那張紙上。兩者都可能讓紅隊的成果歸零，但後者的性質完全不同，它與演習無關。

9.1 | 紅線對防守方同樣成立

紅線通常寫給紅隊看，但它對防守方同樣有約束，而防守方越界的情形在真實演習裏並不少見。常見的有三種。一是為了取證而直接去動攻擊者的機器，這在規則上與攻擊他人系統沒有區別，而且會把一次防禦行動變成一次法律風險；二是為了攔截而把業務系統的策略改得過寬，例如把所有外部訪問直接封掉，這屬於影響業務，與紅隊的拒絕服務在後果上是一回事；三是把演習中看到的個人資料留下來做分析，即使動機是為了改進防護，也觸犯了資料相關的規定。三種的共同點是它們都不是技術判斷失誤，而是忘了紅線是對演習雙方說的。

10 | 小結：流程的骨架

把這一趟走完，紅隊的作業流程可以壓成一句話：先把目標的攻擊面摸清並排出優先級，再從最可行的一個入口打點，把立足點做穩並收集憑據，然後用憑據在內網裏橫向移動，目標是拿到域與域控的權限，期間靠權限維持對抗清理、靠痕跡清理控制暴露，最後把成果整理成舉證與報告交上去。

這條鏈上有四個反覆出現的規律，它們比具體手法更耐時間。第一，入口往往不在防守方重點保護的地方，而在沒有人認領的資產上，因為保護的投入是跟著責任走的。第二，真正讓攻擊擴散的通常不是漏洞，而是可以重複使用的憑據，因為憑據的收益會隨着它被試過的機器數量累加，而漏洞的收益在一次利用之後就結束了。第三，紅隊在內網的真正約束不是技術，是噪音預算，每一步都要在收益與暴露之間取捨，而這個取捨沒有通用答案，它取決於目標單位的監控強度。第四，決定最終得分的除了技術，還有證據能不能通過核驗，這意味着紅隊從第一步開始就要考慮「這一步事後怎麼證明」。

還有一條不在鏈上但貫穿全程的線，是**授權**。它不屬於任何一個階段，沒有它，前面所有階段的性質都會改變。

10.1 | 這條鏈對防守方意味着甚麼

把前面四個規律反過來讀，就是防守方要做的四件事。入口在沒有人認領的資產上，所以盤點與下線比買設備更有效；擴散靠憑據而不是漏洞，所以收斂憑據比修漏洞更能限制損失；紅隊的約束是噪音預算，所以提高監控覆蓋率等於壓縮紅隊的活動空間；得分看證據能不能核驗，所以日誌的保留期與完整性直接決定了紅隊能拿走多少分。四件事都不指向新產品，它們指向的是同一個方向：把已經有的東西弄清楚、守好、記下來。

11 | 想繼續學的話

這一篇寫到了可以據以判斷的顆粒度：每個階段為甚麼做、做的時候看甚麼、做錯了會怎樣、防守方在對面看得到甚麼。但它仍然有意留在一個層次上，也就是機制與判斷，而不是具體工具的操作步驟。

仍然只是骨架的部分有兩塊。一塊是每一種手法的完整利用鏈，本篇只寫到「判斷依據」，沒有寫具體的載荷與參數；另一塊是各類防禦產品的具體

繞過方式，本篇只寫到「繞過的三個方向」，沒有寫每一個方向的實現。這兩塊是有意省略的：它們的內容會隨着產品與版本快速過時，而且需要配合具體環境才成立。

已經另立專題的是流程裏的第一步：外部攻擊面測繪，寫在同目錄的 `recon.typ` 裏。緊接着的站穩那一段也已經展開，寫在同目錄的 [口令與憑據](#) 裏。同一條路上更靠前的那一段也已經展開：從互聯網邊界進去的各類對外服務與設備，寫在同目錄的 [對外服務與邊界設備的漏洞利用](#) 裏。順着同一條路往下，對外服務與邊界設備的漏洞利用、提權、口令與憑據、以及域與內網橫向，都會按同樣的分工另立專題，與本篇共用同一份術語表。

要把它填滿，需要補的基礎大致分四塊。網絡基礎，包括 TCP/IP、HTTP、DNS 與常見的認證協議；Web 安全基礎，包括注入、文件上傳、[反序列化](#) 與 [越權訪問](#)；操作系統與內網，包括 Windows 的認證機制、域的管理方式、以及 Linux 的權限模型；還有工具使用，包括掃描、抓包、代理與命令控制框架。

讀這四塊的時候有一個順序建議：先讀機制，再讀工具。原因是工具會換，而機制在同類系統上一直成立。一個知道認證流程的人，換一套工具只

需要重新熟悉介面；一個只會用工具的人，遇到一個工具不支持的情形就沒有辦法了。

練習的路徑也有講究。公開的靶場與比賽適合練單點技術，它們有明確的題面和反饋；真實環境的手感只能在授權範圍內獲得，企業內部的攻防演練、以及各類向公眾開放的漏洞獎勵計劃，都是合法且有效的場合。無論走哪條路，先確認自己手上的授權範圍，再開始動手。

11.1 | 防守方該學的東西

防守方要補的基礎與紅隊基本重合，差別只在看的方向。網絡基礎、作業系統與內網、以及 Web 安全的機制，防守方要學到能判斷「這個動作在正常情況下會不會發生」的程度，而不是學到能利用的程度。具體的落點有三個：一是協議的正常流程，因為所有異常偵測的前提是知道正常長甚麼樣；二是身分驗證與憑據的存放位置，因為攻擊鏈的中段全在這裏；三是各類系統的日誌裏到底記了甚麼，因為很多單位以為已經記下來了，實際上是沒有開，或者早就被覆蓋了。第三個落點最不起眼，卻往往是復盤時唯一能救命的東西。

12 | 術語表

本表是全文的概念登記區，按閱讀順序排列：基礎概念在前，建立在它之上的概念在後。正文裏每個加下劃線的名詞都指向本表的詞條；讀完一條要回原處，用各自環境的返回鍵即可（瀏覽器或 PDF 閱讀器通常是 Alt+左箭頭）。

12.1 | 護網演習

護網演習是由公安部牽頭組織的實戰化網絡攻防演習，用來檢驗重點單位在真實攻擊下的安全防護水平。參演單位以央企、國企、金融、能源、運營商、醫療、教育等重點行業為主。演習按組織層級分國家級、省級、市級，部分行業也會自行組織同類演習。

它與普通滲透測試的區別有三處。攻擊由多支隊伍同時發起、持續多日、事先不打招呼；攻防雙方都被計分，防守方的處置速度直接影響成績；攻擊目標由組織方指定，不由乙方自己劃定範圍。

組織方一般提前約三週通知參演單位。通知內容包括參演單位要防守的目標系統清單，以及演習的起止時間。

12.2 | 紅隊

紅隊是 **護網演習** 中的攻擊方。紅隊的任務是在演習期內攻下分配給自己的目標，拿到可驗證的證據並提交，以此換取分數。

參加演習的紅隊主要有兩類來源：組織方抽調的專業隊伍，以及參演單位或安全廠商派出的隊伍。一支小隊通常三到五人，常見的分工是一個管情報、一個管突破、一個管內網與收尾。

12.3 | 藍隊

藍隊是 **護網演習** 中的防守方，由參演單位自己的人員和它聘請的安全廠商工程師組成。藍隊的任務有三件：及時發現 **紅隊** 的攻擊、及時處置、並把攻擊鏈還原清楚。三件任缺一件，處置的效果都要打折。

演習期間藍隊二十四小時值守。攻擊不會因為提交窗口關閉而停止，只是成果被留到第二天白天再提交。

12.4 | 紅藍對抗

紅藍對抗指 **紅隊** 與 **藍隊** 分開建制、各自計分的對抗形式。對抗全程有第三方在場。組織方負責提供演習平台、審核 **紅隊** 提交的證據、記錄

雙方的得分與扣分。裁判的裁定在演習中就是最終結果，所以證據的完整性與可核驗性，在紅隊的工作裏和技術突破同樣要緊。

12.5 | 靶標

靶標是演習中紅隊要攻下的目標系統。每支紅隊有分配好的固定目標。組織方另設一批公共目標放進目標池，供所有隊伍爭奪，因此演習一開始，熱門的公共目標往往同時被好幾支隊伍盯着。

靶標一般由參演單位自報，或由組織方指定。紅隊拿下靶標並提交可驗證的證據，經審核通過才算得分。

「拿下」在一份具體的演習規則裏有明確定義，它通常不是「能訪問這個系統」，而是「能以可驗證的方式在靶標系統上執行操作」。紅隊要達成的具體目標因此有三種常見形態：在靶標系統上執行一條指定的命令、取得該系統上指定的資訊、或者取得該系統的管理權限。三種形態的分值不同，而判斷有沒有達成的依據是提交的證據，不是紅隊自己的描述。

12.6 | 演習規則

演習規則由組織方在演習開始前下發，規定計分方式、證據提交窗口與紅線。以紅隊為例，常見的規定是提交窗口在白天若干小時內開啟，證據要能證明權限真實，並且確實來自靶標系統。

紅線是規則裏最要緊的部分：禁止拒絕服務攻擊、禁止刪改業務數據、禁止影響生產系統運行、禁止訪問與演習無關的真實個人數據。越過紅線的隊伍會被扣分，嚴重的直接取消成績。

12.7 | 授權

授權是參演單位或組織方給紅隊的書面許可，寫明可以攻擊的目標範圍、可以使用的時間窗、以及允許造成的影響程度。授權的具體內容由演習規則與組織方的通知共同限定。紅隊的一切動作都以此為邊界。

邊界情形有兩種要分清。一種是授權寫明了範圍，但某一台機器的歸屬有疑問；另一種是歸屬沒問題，但某個動作的影響程度接近上限。兩種情形的處理方式相同：停下、向組織方書面確認、留存回覆。紅隊的通行做法是一律以書面為準，因為口頭同意事後無法證明。

12.8 | 邊界設備

邊界設備指部署在單位網絡與外部網絡之間、用來控制或檢查進出流量的設備。常見的有防火牆、遠端接入設備、入侵偵測與入侵防禦設備、郵件閘道、以及負載均衡。

邊界設備在演習裏的價值有兩重。一是它們自己有管理面，管理面一旦被拿下，紅隊等於直接站在內網門口；二是它們的策略配置本身就是情報，讀一份防火牆規則就等於讀懂目標的網絡分區。

補丁落後是這類設備的常態，成因在正文打點那一章展開。

12.9 | 堡壘機

堡壘機是集中管理運維權限的跳板伺服器。管理員不直接連目標伺服器，而是先登入堡壘機，再由堡壘機轉發到目標，所有操作在堡壘機上留一份會話記錄。

它同時是防守方的審計點與紅隊的高價值目標。堡壘機上通常存着大量被管目標的憑據，拿下它等於一次拿到一整批機器的訪問權；反過來，堡壘機會話記錄是藍隊復盤攻擊路徑時最完整的一份材料。

12.10 | 攻擊面

攻擊面指目標單位暴露在互聯網上、能被外部直接觸達的一切資產與入口。它包括域名與子域名、公網 IP 段、開放端口、對外發佈的 Web 應用與接口、遠程接入通道、以及託管在雲上的服務。

攻擊面裏最危險的部分，是沒有人認領的資產：測試環境、演示系統、開發人員自己搭的服務、上線時登記過而後無人維護的舊後台。它們通常不在運維的巡檢清單上，卻和生產系統連在同一個網絡裏。

12.11 | 內網

內網指目標單位內部、不直接暴露在互聯網上的網絡。辦公網、生產網、管理網都屬於內網。[攻擊面](#)是牆外的部分，內網是牆內的部分，兩者由邊界設備隔開。

內網裏的機器通常不能直接訪問互聯網。[紅隊](#)進入內網之後，原先的攻擊機就聯繫不上目標了，必須另找通道把流量送出來。

12.12 | 信息收集

信息收集是紅隊在動手之前對攻擊面的系統性摸底，也叫偵察。它包括找出目標到底有哪些資產、這些資產分別是甚麼、由誰維護、有沒有已知的問題。

信息收集貫穿整個演習。演習開始前做的是外圍情報；拿到攻擊面上的一個入口之後，還要對內網再做一遍同樣的事。

12.13 | 影子資產

影子資產指單位自己不完全掌握、或根本不知道存在的對外服務。常見來源有四類：開發人員的測試環境、業務上線時臨時開放而後無人維護的入口、移動端與小程序的後端介面、以及併購與分支機構帶進來的舊系統。

這類資產的共同點是沒有人為它負責，所以沒有人給它打補丁、看日誌、做加固。它們是攻擊面上價值最高的部分。

12.14 | 憑證透明度日誌

憑證透明度日誌是公開的、只可追加的 TLS 憑證簽發記錄。憑證頒發機構每簽發一張公開憑證，通常都要把它提交到一個或多個日誌，日誌對任何人不設讀取條件。規範⁶

對紅隊來說，這份日誌的用處是它記着一張憑證裏寫過的所有域名。一次簽發往往把一批相關域名同時寫進記錄，所以查一個已知域名，常常能順出若干個原本不知道的子域名。

它的盲區有兩個：只用自簽憑證、不對外服務的系統不會出現在日誌裏，以及簽發時間早於日誌上線的憑證也不在其中。所以它要和被動 DNS 一類來源互相印證。

12.15 | 被動 DNS

被動 DNS 指長期記錄域名解析結果的歷史數據庫。它不主動查詢，只是把各處觀察到的解析記錄存下來，所以能查到某個域名過去指向過哪些位址。

⁶見 [RFC 9162](#)。

對紅隊的用處是找已經下線但仍有記錄的資產，以及判斷兩個域名的位址是否曾經重合。它與憑證透明度日誌的盲區正好互補：前者管用過的名字，後者管用過的位址。

12.16 | 自治系統號

自治系統號是互聯網路由體系裏標識一個網絡的編號，英文縮寫是 ASN。一個單位、一家運營商、一家雲廠商都可能擁有若干個自治系統號，每個號對應一組可以整體查詢的 IP 段。

它的用處是把「一個域名」還原成「一片網段」。同一個自治系統號下的位址通常屬於同一個組織，所以紅隊可以用它推斷目標自建機房的範圍，也能用它判斷某台機器是不是目標自己的資產。

12.17 | 指紋

指紋是資產在技術上留下的可識別特徵。信息收集走到一定程度，紅隊就能說出對面這台機器裝的是甚麼操作系統、甚麼中間件、甚麼框架、甚麼版本，甚至能猜到它是哪個廠商的哪款設備。這一組判斷就是指紋。

指紋的用處是選武器。知道指紋之後，**紅隊**才知道該翻哪個組件的已知漏洞，不必把所有漏洞都試一遍。

12.18 | 網頁應用防火牆

網頁應用防火牆是檢查 HTTP 請求解析構並攔截攻擊特徵的設備或軟件，英文縮寫是 WAF。它部署在網站前面，按規則匹配請求內容，命中規則就阻斷。

它對紅隊的意義是改變試探的成本。同一個入口在沒有 WAF 時可以反覆試，有 WAF 之後每次嘗試都可能被記錄並觸發攔截。所以紅隊要先認出目標是否有 WAF、是哪一款，再決定用甚麼節奏試。

認它的方法在正文認指紋那一節。

12.19 | 內容分發網絡

內容分發網絡是把網站內容緩存在各地節點的服務，英文縮寫是 CDN。訪問者連到最近的節點，由節點回源或直接作答。

它給防守方帶來一個副作用：從外部解析到的位址是節點的位址，不是目標真實伺服器的位址。所以紅隊若把節點位址當成目標資產去掃，掃到的東西與目標無關，既浪費時間也可能越過 **授權** 邊界。

12.20 | 弱點

弱點指目標身上可以被利用的缺陷，範圍比漏洞寬。它包括軟件漏洞、配置錯誤、弱口令與默認口令、權限設置不當、以及業務邏輯上的設計缺陷。

在 **攻擊面** 上找 **弱點** 靠 **指紋** 指路。同一個弱點在不同階段的價值並不相同：暴露在互聯網上的弱點能換來一個入口，藏在 **內網** 裏的弱點能換來一片區域。

12.21 | 反序列化

序列化是把記憶體裏的物件寫成一段位元組流，反序列化是把這段位元組流還原成物件。這個機制在跨程序傳遞資料時到處都用。

它的危險來自一件事：還原的過程往往會按位元組流裏寫的類型資訊去建立物件，甚至呼叫物件自己的還原方法。於是「能控制位元組流」就等於「能影響對方建立甚麼物件、執行甚麼程式碼」。

危險的觸發點通常在資料要跨程序邊界的地方：工作階段資料、快取、訊息佇列、以及遠端呼叫。這些位置有一個共同特徵，就是資料由一端寫入、由另一端還原，而寫入端未必可信。

12.22 | 越權訪問

越權訪問指使用者做了自己權限範圍之外的事。它分兩類。水平越權是同級別使用者之間互相訪問對方的資料，例如把查詢裏的使用者編號換成別人的；垂直越權是低權限使用者做了高權限的事，例如普通帳號存取了管理介面。

它與其他弱點的差別在於，它通常不需要任何技術鏈，只需要改一個參數或直接訪問一個沒有做檢查的位址。所以它在真實系統裏的出現頻率遠高於外界的預期。

12.23 | 服務端請求偽造

服務端請求偽造指紅隊讓目標伺服器代替自己去訪問一個位址，英文縮寫是 SSRF。觸發點是那些由使用者提供位址、由伺服器去抓取的功能，例如圖片抓取、網址預覽、檔案匯入。

它的價值取決於伺服器能訪問到甚麼。若伺服器能訪問內網位址，這個弱點就從一個功能缺陷變成進入 [內網](#) 的入口，這是它被單獨列出來的原因。

12.24 | 社工釣魚

社工釣魚是從人入手取得訪問權限的手法。常見形式是向目標單位的員工投遞偽裝過的郵件，誘使其點擊鏈接、打開附件、或在仿冒的登錄頁上填寫帳號口令。

[信息收集](#) 階段拿到的員工姓名、崗位與郵箱命名規則，在這一步直接變成投遞清單。釣魚的價值在於它繞開全部邊界設備。防禦方把防火牆、WAF、入侵檢測都部署在互聯網入口，而釣魚郵件進的是員工的收件箱。

12.25 | 誘餌

誘餌指釣魚投遞裏給人看的那一層東西，包括郵件正文、附件、以及連結指向的頁面。它與 **載荷** 是兩件事：誘餌決定對方會不會動手，載荷決定動手之後發生甚麼。

誘餌的成敗取決於它說的事情在對方的工作裏合不合理，與技術關係不大。同一份載荷換一個誘餌帶來的成敗差別，通常大於同一份誘餌換一份載荷。

12.26 | 1day 與 Nday

1day 指剛公開、補丁剛發布不久、多數目標還沒來得及修的漏洞；Nday 指已經公開很久、補丁早已發布、但仍有相當數量目標沒有修的漏洞。兩個詞都是紅隊內部的叫法，不是嚴格的學術術語。

兩者的用法差別在正文漏洞的取捨那一節。要記住的是它們的分界不在漏洞本身，而在目標修沒修。

12.27 | 口令噴灑

口令噴灑指用少量常見口令去試大量帳號，每個帳號只試一兩次。它與逐個帳號窮舉的差別在觸發鎖定的機率。帳號鎖定策略通常按同一帳號連續失敗次數計數，每個帳號只試一次就不會把任何一個帳號試到鎖定。

12.28 | 撞庫

撞庫指拿別處洩漏的帳號口令組合，去試目標單位的系統。它的前提假設是人會在多個系統上重複使用同一套口令。

它與口令噴灑的差別在偵測方式上。撞庫的特徵是大量不同帳號、每個帳號的口令都不同且都失敗；口令噴灑的特徵是大量帳號都試同一個口令。防守方這兩條規則要分開寫。

12.29 | 憑據填充

憑據填充指把已經拿到的憑據拿去試其他系統。它與撞庫的差別在憑據的來源：撞庫用的是別處洩漏的，憑據填充用的是自己在本次行動中取得的。

這一步是內網擴散的主要動力。一次成功的填充就把一台機器的存取權變成一批機器的存取權，所以它的收益遠高於在同一個系統上繼續找漏洞。

12.30 | 多因素認證

多因素認證指登入時要求兩類以上不同性質的憑證，英文縮寫是 MFA。不同性質是關鍵：口令加另一個口令不算多因素，口令加一次性的動態碼或一把私鑰才算。分級與要求⁷

它對紅隊的直接影響是口令那條路被截斷：知道口令也登不進去。但它並不是不可繞，繞法不在於破解第二個因素，而在於讓第二個因素驗證通過的那一刻被別人利用，詳見正文弱口令那一節。

12.31 | 打點

打點指在目標系統上取得第一個可控的執行點。它是紅隊在演習裏真正意義上的開始。在打點成功之前，紅隊做的都是準備。

⁷見 [NIST SP 800-63B-4](#)，該修訂版在二零二五年發佈，取代了此前的 SP 800-63-3。

打點通常走三條路：一是 **攻擊面** 上存在 **弱點** 的 Web 應用與邊界設備，二是 **社工釣魚** 換來的員工終端與帳號，三是供應鏈與第三方運維。三條路的目標一致，都是為了拿到一個立足點。

12.32 | 立足點

立足點是 **打點** 成功之後，**紅隊** 在目標系統上獲得的那個可控的執行點。表現形式可能是服務器上的一個後門文件、一段注入到正常進程裏的代碼、也可能只是一個能用的帳號。

立足點有三個必須解決的問題：能不能執行命令、能不能穩定存活、能不能反覆回來。只滿足第一條的立足點往往活不過半天。

12.33 | 載荷

載荷是紅隊投放到目標系統上、由目標自己執行的那段內容的統稱。它是一個泛稱，具體分三層：**誘餌** 那一層給人看，**加載器** 那一層負責把東西弄起來，**被控端** 那一層長期留在機器上聽 **命令與控制** 指揮。

三層的壽命不同，誘餌投遞一次就廢掉，加載器通常在數小時內被端點防護記住，被控端要活到演習結束。所以三者的投入與更新節奏也不同。

12.34 | 加載器

加載器是 **載荷** 裏第一個跑起來的那一段。它的任務有三件：判斷自己在甚麼環境裏、把真正幹活的內容還原出來、以及換一個已經被信任的宿主去執行它。

它與 **被控端** 的分界在於是否長期存在。幹完就消失的是加載器；留下來維持 **命令與控制** 通道的是被控端。

它的痕跡落在終端上，而且是最容易被記錄下來的那一層，所以它也是三層裏最需要反覆更換的一層。

12.35 | 被控端

被控端是 **載荷** 裏長期留在目標機器上、主動回連控制端並接受指令的那一段。它的功能有五件：回連、取指令、執行、回報、以及自我保護。

它與 **立足點** 的關係是：立足點描述的是紅隊在目標上有一個可控的執行位置，被控端是維持這個位置最常見的實現。後續的取 **憑據**、**提權** 與 **橫向移動** 都從它開始。

它的可見性與存活期成反比：要在目標機器上長期存在，就必須有某些東西是持續的，而持續的東西會在日誌或基線裏形成穩定的痕跡。

12.36 | 代碼簽名

代碼簽名指用一個由受信任機構簽發的證書，為程式檔案附上一段可驗證來源的簽章。作業系統與終端防護會據此調整對這個檔案的初始判斷，包括信譽評級、使用者看到的警告、以及部分策略的放行。

它只影響檔案層這一關，程式跑起來之後的行為該被發現還是會被發現。所以它的作用是降低進門的阻力，不是進門之後的安全。

簽名的痕跡在證書本身：證書有實名審核、有簽發紀錄，被濫用之後會被吊銷，因此紅隊用哪一類證書是一個要事先想清楚的決定。

12.37 | 沙箱

沙箱指防守方或分析平台用來在隔離環境裏執行可疑檔案、並記錄其行為的系統。它與真實目標環境的差別在於：沒有真實使用者在用、硬體與系統設定較為固定、以及執行時間通常有限。

對紅隊的意義有兩重。一是它會記錄 **載荷** 在隔離環境裏的完整行為，於是同一版載荷用第二次就會被認出來；二是它能被用來排除明顯有問題的載荷，因為在沙箱裏過得了不代表目標上過得了，沙箱裏過不了則幾乎可以確定目標上也過不了。

12.38 | 重定向器

重定向器是 **命令與控制** 架構裏的一台中間機器。**被控端** 連的是它，它再把流量轉給真正處理指令的控制端。

它的作用是讓被發現時的損失變小，而不是讓自己被發現不了：防守方順著流量查到的是這台機器，上面沒有任何東西，處置它只讓紅隊損失一跳；同時因為被控端記住的位址沒變，紅隊可以在不換載荷的情況下更換後端。

代價是多了一跳，而且它的位址遲早會被查出來。

12.39 | 命令與控制

命令與控制指紅隊與立足點之間傳遞指令和數據的通道，英文縮寫是 C2。立足點只解決「能執行一次」的問題，命令與控制解決的是「能一直指揮」的問題。

一條合格的 C2 通道要能扛住中斷、要能換協議換端口、要能藏進正常流量裏。演習中常見的做法是把 C2 流量混進 HTTPS 或 DNS 查詢，因為這兩類流量在企業網絡裏幾乎不可能被完全封死。

12.40 | 域前置

域前置指在一條 TLS 連線上，讓網絡中間看到的主機名與實際處理請求的主機名不一致。中間設備只能看到加密連線外層的那個名字，所以按名字做的封鎖會失效。

它靠的是大型內容分發網絡同時承載大量客戶域名的特性。這條路的可用性取決於服務商的策略，近年多數大廠已明確禁止，所以它是一種會過時的手法，不是穩定通道。

12.41 | 隱蔽通道

隱蔽通道指把 [命令與控制](#) 流量藏進看起來正常的流量裏。選哪種流量做載體，判斷標準是它在目標網絡裏是否必須放行。常見載體是 HTTPS、DNS 查詢、以及各類雲服務與協作平臺的介面。

MITRE ATT&CK 把這類做法歸在應用層協議與代理兩族，⁸

12.42 | 心跳與抖動

心跳指被控端按固定間隔回連控制端，用來確認通道還在、順便取回指令。抖動指把這個間隔加上隨機量，讓回連的時間點不呈現固定週期。

為甚麼要抖動：固定週期的回連在流量統計上是一個明顯的節律，很容易被基線模型挑出來；加了隨機量之後，單看時間間隔就沒有可用的特徵。代價是指令的下發延遲變大。

⁸見 [T1071](#) 與 [T1090](#)。

12.43 | 進程注入

進程注入指把一段程式碼寫進另一個已經在運行的進程，讓它以那個進程的身份執行。常見做法是在目標進程裏分配記憶體、寫入內容、再讓它開一條執行緒去跑。

紅隊用它有兩個目的：一是借用一個本身被信任的進程，二是讓自己的程式碼不出現在磁碟上。它的痕跡主要留在跨進程的記憶體操作與執行緒建立上，端點防護正是據此偵測的。

12.44 | 白利用

白利用指不帶自己的程式，直接用系統自帶的、本來就有正當用途的工具完成動作。腳本宿主、計劃任務工具、遠端管理工具、以及各類系統管理程式都屬於這一類。

這種做法的難點在防守方：被呼叫的工具是系統的一部分，按檔案信譽攔不住。可用的工具清單有公開整理，⁹

⁹見 [LOLBAS](#)。

12.45 | 端點偵測與回應

端點偵測與回應是裝在終端上、記錄行為而不只比對檔案特徵的防護軟件，英文縮寫是 EDR。它記程序的父子關係、跨進程操作、以及命令列內容，所以對白利用與進程注入這類做法比傳統防毒更敏感。

12.46 | 提權

提權指把已經到手的低權限帳號提升為管理員或系統權限。[立足點](#) 往來得並不好看。Web 服務以普通用戶身份運行，拿下的帳號只能訪問一個目錄。提權就是把這個身份換成能讀寫全盤的那一個。

提權成功的直接收益有兩個：一是能動系統配置，二是能讀到本機保存的帳號口令與密碼哈希。後者比前者重要得多。

12.47 | 提權面

提權面指一台機器上一切能讓低權限身份取得更高權限的地方。它的範圍比[弱點](#) 寬，既包括設定錯誤與權限配置不當，也包括本身合法、卻被授予了超出需要的權力的功能。

一條提權面要同時滿足三個條件： 有一個會以更高權限運行的主體，這個主體會去讀取或執行某個東西， 而那個東西當前身份能改。 三個條件缺一個， 這條路就不通。

紅隊清點提權面時按主體的名目走： 服務、 計劃任務、 核心驅動、 定時任務、 以及被授予額外權力的檔案。 這幾類在兩個操作系統上的叫法不同， 結構相同。

12.48 | 服務設定缺陷

服務設定缺陷指 Windows 服務的設定、 或它所依賴的路徑， 落在當前身份可改的範圍裏。 服務通常以系統身份運行， 所以改動服務設定或它所執行的檔案， 等於讓系統身份替紅隊執行。

常見情形有三種： 服務可執行檔的路徑沒有加引號， 而路徑上有一段目錄可寫； 服務可執行檔所在的目錄可寫； 以及服務物件本身的權限設定允許普通使用者改設定。

它與目錄權限那一類同源， 差別在改的對象不同： 這一類改的是服務指向哪一個檔案。

12.49 | 目錄權限缺陷

目錄權限缺陷指一個高權限程序在運行時會去讀取或執行的位置，落在當前身份可寫的目錄裏。程序按名字去找檔案時，若搜尋順序上有一段目錄可寫，紅隊在那裏放一個同名的檔案，程序就會讀到或執行到它。

它與 **服務設定缺陷** 同源，都是高權限主體碰到了低權限身份能改的東西。差別在這一類不必改任何設定，只要那個位置可寫。

判斷的關鍵不是目錄可不可寫，而是程序會不會真的去那裏找。

12.50 | 核心驅動

核心驅動是運行在 Windows 核心態的程式，它的權限與作業系統同級。普通使用者無權載入驅動，所以這一類提權面要靠兩條路繞過去。

一條是用一個已簽章但有缺陷的驅動：簽章讓它通過檢查，缺陷讓它替紅隊做事。另一條是利用核心本身的漏洞，在核心態執行任意程式碼。

它與 **提權面** 裏其他幾類不同。其他幾類是設定被人用錯了，這一類本質上是 **弱點**，需要版本匹配，而且一旦失手會直接把機器帶下線。

12.51 | 令牌

令牌是 Windows 在一個程序登入之後發給它的身份憑證。令牌裏寫着這個身份是誰、屬於哪些群組、以及持有一組甚麼特權，後續每一次操作都憑令牌判斷權限。

令牌分兩種。主令牌代表登入的那個使用者；模擬令牌代表被模擬的另一個身份，一個執行緒可以在模擬狀態下以別人的身份操作。正因為有模擬，拿到一個高權限的令牌就等於拿到了那個身份。

12.52 | 令牌竊取

令牌竊取指把一個高權限程序持有的 **令牌** 據為己有，從而以那個身份操作。

它與 **提權** 的關係是手段與目的。提權的目標之一是拿到更高權限的令牌，而拿到令牌之後往往就不必再做提權動作了。

12.53 | 特殊權限位元

特殊權限位元指可執行檔上的一組標記，最要緊的兩個是 `setuid` 與 `setgid`。一個帶 `setuid` 的檔案被執行時，執行的身份是檔案的擁有者，而不是呼叫它的那個使用者。

系統裏本來就有一些必須帶這個位元的程式，所以問題不在位元本身，而在一個以更高身份運行、又能被使用者影響其行為的程式：它會讀取使用者給的檔案、接受使用者給的參數、或者去呼叫使用者能控制的另一個程式。

它與能力位元那一類講的是同一個問題的兩種做法，差別在位元是全是全無地換一個身份，能力是把高權限拆成一項一項單獨授予。這一類是 **提權面** 的一種。

12.54 | 命令提權工具

命令提權工具指讓指定使用者以指定身份執行指定命令的工具，在 Linux 上最常見的是 `sudo`。規則寫在一份設定檔以及它引入的目錄裏，每一條規則說明誰、可以以誰的身份、執行哪一條命令。

它成為提權面的原因有三處。被允許執行的命令本身能執行別的命令或開一個 shell；規則用了通配符或前綴比對，實際允許的範圍比寫出來的大；以及規則保留了呼叫者的環境變數，而某些變數會被程式讀取。

還有一類設定是不要求口令，它降低的不是權限，是拿到權限的門檻。這一類是 **提權面** 的一種。

12.55 | 定時任務

定時任務是 Linux 按時間表自動執行命令的機制，常見形態有 cron、一次性的 at、以及以服務管理員為基礎的定時器。

它與 Windows 上的計劃任務是同一件事的兩種實現，缺陷的形態也相同：被執行的腳本本身可寫、腳本所在目錄可寫，或者時間表的定義檔本身可寫。

它另有一處設定特有的問題：定時任務執行時的搜尋路徑比互動式 shell 短，腳本裏若按名字呼叫外部命令，在被搜尋的目錄裏放一個同名程式就能把呼叫接過來。

12.56 | 核心漏洞

核心漏洞指作業系統核心裏的缺陷，利用它可以直接取得系統的最高權限。它的成因類型包括核心介面對使用者傳入的參數校驗不足、競態條件、以及記憶體管理上的錯誤。

它與 **核心驅動** 是核心態這一類提權面在兩個系統上的兩個名字。與其他幾類相比，這一類的成本與風險都高：需要版本匹配，利用過程不穩定，失敗的後果往往是機器崩潰或重啓。

所以在紅隊的次序裏，它排在最後，只在其他路走不通時才考慮。

12.57 | 能力位元

能力位元是 Linux 把最高權限拆成一項一項之後的產物。原本只有最高權限與普通使用者兩種身份，有了能力位元之後，可以單獨把某一項權力授予一個程式，例如讀取任意檔案、切換使用者身份、或管理網路設定。

它成為提權面的原因是：能力集裏有幾項單獨就等於完整的控制權，而被授予這些能力的檔案，往往是廠商或運維為了省事加上去的。

它與 [特殊權限位元](#) 解決的是同一個問題，差別在授予的粒度。兩者的風險相同，都是把不需要的權力給了出去。

12.58 | 憑據轉儲

憑據轉儲指把作業系統與相關服務保存在本機的憑據取出來，包括記憶體快取中的身份資訊、本機帳號資料庫裏的口令雜湊、以及各類服務自己存的憑據。

它是 [提權](#) 之後最優先做的動作，因為一台機器上的憑據往往能用在一批機器上。MITRE ATT&CK 把這類做法歸在 T1003，¹⁰

12.59 | 憑據

憑據指證明身份的東西，包括帳號與口令、口令的哈希值、Kerberos 票據、以及配置文件與腳本裏明文保存的密碼。[立足點](#) 是位置，憑據是鑰匙。

憑據比漏洞值錢，原因有三個。一個漏洞通常只在一台機器上有效，而憑據可以用在目標單位的許多系統上；不少口令在 [內網](#) 裏被多台機器共

¹⁰ 見 [OS Credential Dumping](#)。

用；用憑據登錄產生的流量完全合法，防守方很難從協議層面把它和正常運維區分開。

12.60 | 域與域控

域是 Windows 網絡中集中管理帳號與權限的目錄服務，域控是運行這套服務的服務器。在企業內網裏，域控保存着全網的帳號、口令哈希、以及誰能用哪台機器的授權規則。

對紅隊來說，內網裏的域控是最有價值的一台機器。拿到憑據之後，絕大多數橫向移動都要經過域的認證機制。控制域控，等於拿到整個內網的萬能鑰匙。

12.61 | 票據

票據是域與域控環境裏用來證明身份的憑證。使用者通過認證之後拿到一張票據，之後訪問域內資源時出示這張票據，不必每次都重輸口令。

對紅隊來說票據有兩個特點。一是它可以直接被使用，不需要還原成口令；二是它有有效期，過期之後要重新取得，所以拿到票據之後的動作要在有效期內完成。

12.62 | 橫向移動

橫向移動指紅隊從已有的機器出發，用同樣的憑據去控制內網裏更多機器的過程。它是演習中持續時間最長、最能體現技術水平的環節。

橫向移動很少靠新漏洞，多數時候靠的是目標自己的管理工具：遠程桌面、文件共享、遠程過程調用、計劃任務、以及各種自動化運維腳本。這些工具本來就是為了方便管理員批量操作而存在的。

12.63 | 隧道與代理

隧道與代理是紅隊在自己與內網之間搭的一條中轉通道。內網裏的機器通常不能直接訪問互聯網，紅隊的攻擊機也在外面進不來，兩邊需要一個中間人替它們轉發流量。

常見做法是在已經控制的邊緣機器上開一個監聽端口，把內網的流量包進合法的出網請求裏送到攻擊機，或者讓攻擊機的指令通過這條通道分發到內網各處。這條通道通常與命令與控制通道一起維護。隧道打通與否，直接決定紅隊能走多遠。

12.64 | 隔離區

隔離區指介於外網與 [內網](#) 之間、用來放置對外服務的那一段網絡，也叫 DMZ。對外提供服務的伺服器放在這裏，它與真正的內網之間再隔一道策略。

紅隊拿下隔離區裏的機器並不等於進入內網，兩者之間往往還有一次跨區跳躍。這也是為甚麼從 [邊界設備](#) 進去的收益比從對外業務系統進去更高，邊界設備本身常常同時連着兩個區。

12.65 | 噪音預算

噪音預算指一次行動允許在目標那裏留下多少可被發現的痕跡。它是紅隊自己給自己定的約束，不是防守方的概念。

為甚麼需要這個約束：在互聯網上做情報收集時，掃描量大、被發現也無所謂；進到 [內網](#) 之後，一次全網段掃描就可能直接引來處置，把已經拿下的 [立足點](#) 賠進去。所以內網的每一步都要先算這一步要花掉多少噪音、自己還剩多少。

12.66 | 組態管理工具

組態管理工具指用來批量下發設定與執行腳本到大量機器的運維系統。這類工具在設計上就要能在一台機器上對很多機器下命令，所以它天然具備橫向執行的能力。

紅隊在意它還有一個原因：它的設定檔與下發的腳本裏常常直接寫着高權限帳號。拿到這些檔案等於同時拿到橫向通道與憑據。

12.67 | 特權帳號

特權帳號指在域內或在本機上擁有管理權限的帳號，包括域管理員、本機管理員、以及各類服務帳號。

它們是內網階段的首要目標，原因是 **憑據** 讓攻擊者可以在大量機器上活動，而特權帳號本身的活動範圍往往就直接覆蓋一整片網絡。

12.68 | 權限維持

權限維持指 **紅隊** 在已經拿下的機器上留下能長期反覆回來的入口。演習期間 **藍隊** 會不斷排查，被發現的 **立足點** 會被清掉，所以 **紅隊** 在關鍵機器上通常不止留一條路。

常見的落腳點包括新增的本地帳號與域帳號、計劃任務、系統服務、以及各類 Web 中間件的後門。留在 [域與域控](#) 上的入口價值最高，因為它同時意味着整片 [內網](#) 的訪問權。

12.69 | 痕跡清理

痕跡清理指 [紅隊](#) 在收尾時處理自己留下的記錄，也叫打掃戰場。要處理的東西包括上傳的工具與樣本文件、新增的帳號與計劃任務、被修改的配置、以及系統日誌與安全設備日誌裏的相關條目。

清理的目的首先是保住入口，避免 [立足點](#) 與 [權限維持](#) 的線索被 [藍隊](#) 順着查出來。它與計分的關係是另一回事，兩者之間的取捨寫在正文站樁與打掃那一節。

12.70 | 蜜罐

蜜罐指防守方故意放置的、看起來有價值但實際無人使用的系統或帳號。它不對外提供業務，所以任何對它的訪問都是異常的。

它的價值不在攔截，在於發現。一個蜜罐被觸碰，藍隊立刻知道有人已經進來了，並且能拿到攻擊者的手法與來源。對紅隊來說，過於容易拿下本身就是一個應當放棄的信號。

12.71 | 舉證與報告

舉證指紅隊把拿下的權限整理成組織方可以核驗的證據，再做提交。證據通常包括靶標系統上的命令執行截圖、系統信息頁面、以及能證明操作發生在演習時間窗內的記錄。演習規則對提交窗口有明確規定，過了窗口的成果只能留到下一次。

報告是演習結束後交的另一份東西，內容包括攻擊路徑、用到的弱點、以及藍隊在哪些環節本可以攔住卻沒有攔住。紅隊的報告與藍隊的復盤合起來，才是一次演習對參演單位的真正價值。